



#inc

Ajouter son propre #include

Ranger ses fonctions dans un fichier, et en faire entrer dans la console

Ce document explique, du tout début jusqu'au bout, comment fonctionnent les lignes `#include` de l'atelier Game Boy, et comment en **ajouter une à toi**.

On y suit une seule fonction, `bande()`, qui fait tout le voyage : écrite dans le programme, rangée dans un fichier voisin, puis devenue une fonction de la console que l'on demande par `#include <bande>`.

Tout le code du compilateur montré ici est lu dans les vrais fichiers du projet ; chaque message d'erreur est celui que le compilateur donne vraiment ; chaque écran est celui que le programme affiche dans l'émulateur. Pour remettre ce document à jour : `node outils/pdf-include.mjs`.

Sommaire

- 0** Les bases : tout ce qu'il faut savoir avant • débutant
 - 0.1 Ce document, et comment t'en servir
 - 0.2 Un fichier, un dossier, une extension
 - 0.3 Un programme, un compilateur, une cartouche
 - 0.4 L'octet, et la place dans la cartouche
 - 0.5 L'écran : des cases, des tuiles, la police
 - 0.6 Une fonction : la définir, l'appeler
 - 0.7 La boucle for, tour par tour
 - 0.8 La ponctuation du C++
 - 0.9 L'atelier : où écrire, où regarder
 - 0.10 Les touches utiles, et l'éditeur de texte
 - 0.11 La fenêtre de commande
 - 0.12 git : garder une trace, revenir en arrière
- A** Les deux sortes d'#include • débutant
- B** Ce qui se passe quand on compile •• intermédiaire
- C** Le chapitre du parcours, étape par étape •• intermédiaire
- D** Le JavaScript qu'il faut pour la suite •• intermédiaire
- E** La recette : ajouter ta fonction à la console ••• avancé
- F** Les erreurs, et ce que le compilateur répond •• intermédiaire
- G** Ce que ça coûte dans la cartouche ••• avancé
- H** Les questions qu'on se pose •• intermédiaire
- I** L'aide-mémoire, à garder sous les yeux •• intermédiaire
- J** Tous les #include de la console • débutant
- K** Le lexique : chaque mot technique • débutant

Comment lire ce document

Tu débutes ? Lis la **Partie 0** en entier, puis les parties A et C. Reviens aux autres plus tard, quand tu voudras aller plus loin : rien ne presse.

Tu as déjà suivi le parcours de l'atelier ? Survole la Partie 0, lis A, B et C, puis D et E pour ajouter ta propre fonction à la console.

Chaque partie dit son niveau (• débutant, •• intermédiaire, ••• avancé) et ce qu'il faut avoir lu avant. **Un mot que tu ne connais pas ?** Le lexique (partie K) le définit et renvoie là où il est expliqué.

Si tu n'as que deux minutes

`#include "outils.cpp"` (des **guillemets**) colle **ton** fichier dans le programme. Tu le crées avec le bouton « + **fichier** » de l'atelier.

`#include <bande>` (des **chevrons**) demande une **fonction de la console**, qui existe déjà dans le compilateur.

Pour ajouter une fonction à la console : son code dans `compilateur/emetteur.js`, son nom dans `compilateur/inclusion.js`, son aide dans `aide-fonctions.js`. Puis **Ctrl+F5** dans le navigateur.

Les bases : tout ce qu'il faut savoir avant

• **débutant** aucune connaissance demandée en dehors de ce qui est cité

Chaque section ci-dessous explique une seule notion, en partant de zéro. Les autres parties y renvoient : « À lire avant : 0.6 » veut dire « lis la section 0.6 si tu ne sais pas encore ce qu'est une fonction ».

0.1 Ce document, et comment t'en servir

Le document répond à une question : « comment ajouter moi-même un `#include` ? ». Pour y répondre vraiment, il faut quelques bases : ce qu'est un fichier, un programme, une fonction, un octet... Cette Partie 0 les explique toutes, **en partant de zéro**. Si tu les connais déjà, survole-la.

Le code est écrit dans des cadres gris, avec une police où toutes les lettres ont la même largeur. Les mots qui viennent du code, dans le texte, sont écrits `comme ceci`.

Les commentaires : dans le code, tout ce qui suit `//` sur une ligne est une explication pour l'humain. La console l'ignore. Ce document en met beaucoup : lis-les, ils font partie de l'explication.

Les flèches `→` veulent dire « donne » ou « devient » : `2 + 3 → 5`.

Un conseil

Garde l'atelier ouvert à côté de ce document, et **essaie chaque programme**. Lire un programme et le voir tourner, ce n'est pas du tout la même chose : on comprend dix fois mieux en changeant un nombre et en regardant ce qui se passe.

0.2 Un fichier, un dossier, une extension

Un fichier est un document rangé sur l'ordinateur : un texte, une image, un programme. Il a un **nom**, par exemple `inclusion.js`.

L'extension est la fin du nom, après le point. Elle dit ce que contient le fichier :

<code>.cpp</code>	un programme écrit en C++ (le langage de tes programmes Game Boy)
<code>.h</code>	un « en-tête » C++ : un fichier fait pour être collé en haut d'un autre
<code>.js</code> <code>.mjs</code>	un programme écrit en JavaScript (le langage de l'atelier et du compilateur, voir partie D)
<code>.html</code>	une page web, qu'un navigateur affiche
<code>.pdf</code>	un document à lire ou imprimer, comme celui-ci
<code>.md</code>	un texte simple, avec un peu de mise en forme
<code>.gb</code>	une cartouche Game Boy : ce que le compilateur fabrique

Un dossier est une boîte qui contient des fichiers et d'autres dossiers. Un **chemin** dit où trouver un fichier en ouvrant les boîtes l'une après l'autre, séparées par `/` : `compilateur/inclusion.js` veut dire « le fichier `inclusion.js`, dans le dossier `compilateur` ».

Le **dossier du projet** est celui qui contient `index.html` (la page de l'atelier). Tous les chemins de ce document partent de lui. Ceux dont on se sert :

```
le dossier du projet/
├─ index.html          la page de l'atelier
├─ aide-fonctions.js   l'aide de l'éditeur (partie E, étape 5)
├─ compilateur/       le compilateur : il transforme ton C++ en cartouche
│   ├── inclusion.js   les #include et la liste des fonctions de la console
│   ├── analyseur.js   la lecture du programme
│   └─ emetteur.js     la traduction, et les fonctions de la console
├─ tuto/              les leçons du parcours
│   ├── lecons.js      les leçons (et les tutos des fonctions)
│   └─ programmation.js le cours (dont le chapitre « Tes propres #include »)
├─ outils/            des petits programmes : compiler, faire les PDF...
└─ documents/         les documents, dont ce PDF
```

Dans l'atelier, un « onglet » de l'éditeur est un fichier : `principal.cpp` est le fichier principal de ton programme ; le bouton « + fichier » en crée un autre, à côté (section 0.9).

0.3 Un programme, un compilateur, une cartouche

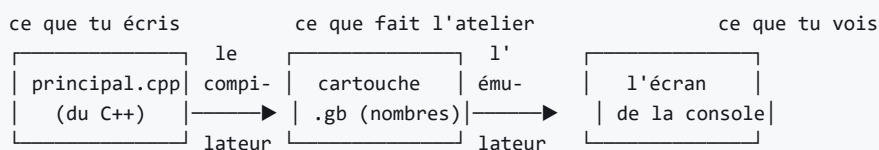
Un **programme** est une suite d'ordres écrits pour une machine. Tu l'écris en **C++**, un langage fait pour être lu par des humains : `poser(2, 5, ALPHABET[0]);` se comprend presque comme une phrase.

Le **processeur de la Game Boy ne comprend pas le C++**. Il ne comprend que des **nombres** : chaque nombre est un ordre minuscule (« copie cette case », « ajoute 1 », « saute là-bas »).

Le **compilateur** est le traducteur : il lit ton C++ et écrit les nombres que le processeur comprend. Dans l'atelier, il tourne tout seul dès que tu t'arrêtes d'écrire.

La **cartouche** est le résultat : un fichier `.gb` qui contient tous ces nombres, plus les dessins et la musique. C'est l'équivalent de la cartouche en plastique qu'on mettait dans une vraie Game Boy.

L'**émulateur** est une Game Boy imitée par l'ordinateur : il lit la cartouche et fait exactement ce qu'une vraie console ferait. C'est l'écran que tu vois dans l'atelier.



Une image pour retenir

Le C++ est une **recette** écrite en français. Le compilateur est un **traducteur** qui la réécrit dans la langue du cuisinier. La cartouche est la recette traduite. L'émulateur est le **cuisinier** qui la suit, à la lettre.

0.4 L'octet, et la place dans la cartouche

Un **octet** est une **petite case de mémoire qui retient un nombre de 0 à 255**. Pas plus : 256 valeurs possibles. C'est l'unité de base de toute la console.

Pourquoi 255 ? Un octet est fait de **8 bits**, 8 interrupteurs qui valent 0 ou 1. Huit interrupteurs donnent $2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 256$ combinaisons : de 0 (tout éteint) à 255 (tout allumé).

Ce qui se passe au-delà : $255 + 1$ ne donne pas 256, mais **0** — le nombre « fait le tour », comme un compteur kilométrique. Et $0 - 1$ donne 255.

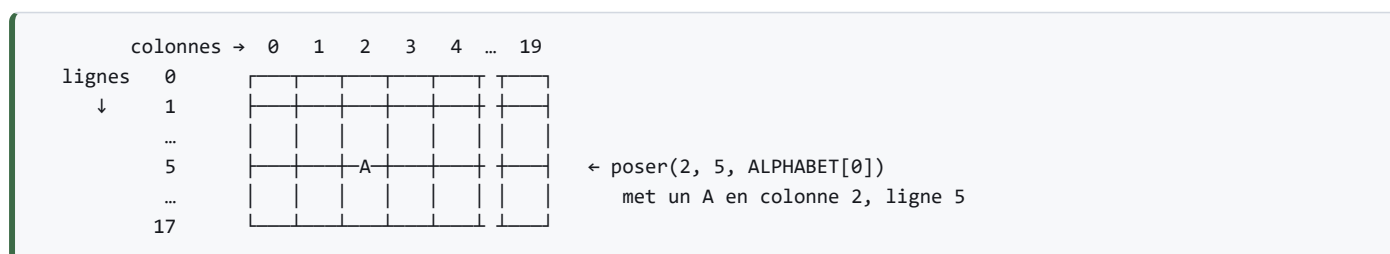
Une cartouche de l'atelier contient 32 768 octets (on dit « 32 Ko », 32 kilo-octets). Tout le jeu doit tenir dedans : le programme traduit, les dessins, la musique. C'est **très peu** : une seule photo de téléphone en prend des centaines de fois plus.

C'est pour cela que la console a la règle des `#include` : chaque fonction de la console prend des octets ; on ne grave que celles qu'on demande (partie A) et qu'on appelle vraiment (partie G).

0.5 L'écran : des cases, des tuiles, la police

L'écran de la Game Boy mesure 160×144 points (des « pixels »). On ne le dessine pas point par point : il est découpé en **cases de 8×8 pixels**. Cela fait **20 colonnes** (numérotées de 0 à 19, de gauche à droite) et **18 lignes** (de 0 à 17, de haut en bas).

Une case se désigne par deux nombres : sa colonne, puis sa ligne. La case (0, 0) est en haut à gauche ; la case (19, 17) en bas à droite ; (2, 5) est la 3e colonne de la 6e ligne (on compte à partir de 0).



Une tuile est le dessin d'une case : 8×8 pixels, chacun dans une des 4 nuances de la console. Une tuile prend **16 octets** de cartouche (2 octets par rangée de 8 pixels, et 8 rangées).

La police est l'ensemble des tuiles des lettres : A à Z, les chiffres 0 à 9, l'espace et quelques signes (43 en tout). Elles sont rangées dans un tableau nommé `ALPHABET` : `ALPHABET[0]` est le A, `ALPHABET[1]` le B, `ALPHABET[2]` le C... `ALPHABET[23]` le X. Le nombre entre crochets est la **place** dans le tableau, en partant de 0.

`poser(colonne, ligne, tuile)` met une tuile dans une case. C'est la fonction de la console dont se sert tout ce document.

0.6 Une fonction : la définir, l'appeler

Une fonction est un geste auquel on donne un nom, pour pouvoir le refaire sans le réécrire. Il y a deux moments bien différents :

```
// 1. LA DÉFINIR : écrire, une fois, ce que fait le geste.
void saluer(uint8_t ligne) {           // son nom : saluer ; son paramètre : ligne
    poser(0, ligne, ALPHABET[7]);      // un H...
    poser(1, ligne, ALPHABET[8]);      // ...et un I, sur la ligne demandée
}

// 2. L'APPELER : faire le geste, autant de fois qu'on veut.
saluer(3);    // HI sur la ligne 3
saluer(10);   // HI sur la ligne 10
```

Chaque mot de la première ligne a un rôle :

`void`

« vide » : la fonction ne **rend** rien. Elle agit (elle pose des lettres), c'est tout.

<code>saluer</code>	son nom . C'est lui qu'on écrit pour l'appeler.
<code>(uint8_t ligne)</code>	son paramètre : une case de mémoire nommée <code>ligne</code> , qui sera remplie au moment de l'appel. <code>uint8_t</code> est son type : un octet, un nombre de 0 à 255 (section 0.4).
<code>{ ... }</code>	les accolades entourent le corps : les ordres exécutés à chaque appel.

Paramètre et argument, deux mots pour deux choses. Le **paramètre** est la case vide dans la définition (`ligne`). L'**argument** est la valeur qu'on met dedans à l'appel (`3`). Pendant `saluer(3)`, `ligne` vaut 3 ; pendant `saluer(10)`, elle vaut 10.

Plusieurs paramètres se séparent par des virgules, et l'appel donne les arguments **dans le même ordre** : `bande(colonne, ligne, tuile, longueur)` s'appelle `bande(2, 5, ALPHABET[0], 10)` → colonne = 2, ligne = 5, tuile = le A, longueur = 10.

Rendre une valeur : une fonction peut aussi calculer un résultat et le rendre avec `return`. Elle s'écrit alors `uint8_t` au lieu de `void` : `uint8_t double_de(uint8_t n) { return n + n; }` → `double_de(4)` vaut 8.

`main()` est la fonction par laquelle tout programme commence. La console l'appelle toute seule à l'allumage.

Une fonction de la console (`poser`, `texte`, `bande`...) est une fonction comme les autres, sauf qu'elle a été écrite **par le projet**, pas par toi. C'est exactement pour cela qu'on peut en ajouter une (partie E).

0.7 La boucle for, tour par tour

Une boucle répète des ordres. La boucle `for` les répète en comptant. Elle a trois morceaux entre ses parenthèses, séparés par des points-virgules :

```
for (uint8_t i = 0; i < 4; i++) {
    // départ      L tant que L après chaque tour L
    poser(2 + i, 5, ALPHABET[0]); // le corps : répété à chaque tour
}
```

départ	<code>uint8_t i = 0</code> : crée un compteur nommé <code>i</code> , qui vaut 0. Fait une seule fois , au début.
tant que	<code>i < 4</code> : la condition. Vérifiée avant chaque tour : si elle est vraie, on fait le tour ; si elle est fausse, la boucle s'arrête.
après chaque tour	<code>i++</code> : ajoute 1 à <code>i</code> . (On peut écrire <code>i = i + 2</code> pour avancer de 2.)

Déroulé, tour par tour :

Tour	i vaut	i < 4 ?	Ce qui est fait
1	0	oui	<code>poser(2, 5, A)</code> puis i devient 1
2	1	oui	<code>poser(3, 5, A)</code> puis i devient 2
3	2	oui	<code>poser(4, 5, A)</code> puis i devient 3
4	3	oui	<code>poser(5, 5, A)</code> puis i devient 4
—	4	non	la boucle s'arrête : 4 A posés, colonnes 2 à 5

À retenir : une boucle `for (i = 0; i < N; i++)` fait **exactement N tours**, avec `i` qui vaut 0, 1, 2... jusqu'à N - 1. C'est le cœur de `bande()` : `N` est la longueur.

0.8 La ponctuation du C++

<code>;</code>	le point-virgule termine un ordre, comme un point termine une phrase. L'oublier est l'erreur la plus fréquente.
<code>{ }</code>	les accolades regroupent plusieurs ordres : le corps d'une fonction, d'une boucle, d'un <code>if</code> .
<code>()</code>	les parenthèses entourent les arguments d'un appel, les paramètres d'une définition, la condition d'un <code>if</code> ou d'une boucle.
<code>[]</code>	les crochets désignent une place dans un tableau : <code>ALPHABET[0]</code> .
<code>,</code>	la virgule sépare les arguments : <code>poser(2, 5, A)</code> .
<code>// ...</code>	un commentaire : jusqu'au bout de la ligne, ignoré par la console.
<code>"..."</code>	des guillemets entourent un texte : <code>texte(1, 1, "SALUT")</code> . Aussi le nom d'un fichier dans <code>#include "outils.cpp"</code> .
<code>< ></code>	des chevrons : « plus petit que » dans <code>i < 4</code> ; et le nom d'une fonction de la console dans <code>#include <bande></code> .
<code>=</code> et <code>==</code>	<code>x = 3</code> range 3 dans x. <code>x == 3</code> demande si x vaut 3 (vrai ou faux).
<code>#</code>	une ligne qui commence par <code>#</code> n'est pas un ordre pour la console, mais une consigne pour la préparation du programme : <code>#include</code> .

0.9 L'atelier : où écrire, où regarder

L'atelier est la page `index.html`, ouverte dans un navigateur. On le lance en double-cliquant `lancer.bat` dans le dossier du projet ; il s'ouvre à l'adresse `http://localhost:8000`.

En haut, des boutons de mode :

-  **MODE CRÉATION** : dessiner les tuiles, composer la musique, à la souris — **et écrire le programme**, dans l'éditeur juste au-dessus des ateliers. Le bouton « ↗ **Agrandir le code** » donne tout l'écran au programme ; « ↶ **Revoir les ateliers** » les fait revenir. C'est là qu'on se sert des `#include`.
-  **APPRENDRE** : le parcours de leçons. Le chapitre « Tes propres `#include` » est le dernier.
-  **MODE MACHINE** : ce que le processeur exécute vraiment.
-  **LES SOURCES** : tous les documents, dont celui-ci.

L'éditeur du programme montre un **onglet par fichier**. Au départ, il n'y a que `principal.cpp`. Le bouton « + fichier », au-dessus de l'éditeur, en ajoute un : tu lui donnes un nom (par exemple `outils.cpp`), et il apparaît comme un nouvel onglet. Cliquer un onglet l'ouvre.

La console se met à jour toute seule : dès que tu t'arrêtes d'écrire, le programme est recompilé et relancé. Si quelque chose ne va pas, le message du compilateur s'affiche sous l'éditeur.

0.10 Les touches utiles, et l'éditeur de texte

Pour la partie E, on modifie des fichiers du projet lui-même, en dehors de l'atelier. Il faut un **éditeur de texte** : un programme qui ouvre un fichier, le montre tel quel, et l'enregistre. Le **Bloc-notes** de Windows suffit (clic droit sur le fichier → « Ouvrir avec » → Bloc-notes). **Visual Studio Code** (gratuit) est bien plus confortable : il colore le code et numérote les lignes.

Attention : pas Word

Un traitement de texte (Word, LibreOffice) ajoute de la mise en forme invisible et remplace les guillemets droits par des guillemets courbes : le fichier ne marcherait plus. Toujours un **éditeur de texte**.

Touches	Où	Ce qu'elles font
Ctrl+F	éditeur, navigateur	chercher un mot dans le fichier ou la page (« Trouver »)
Ctrl+G	Visual Studio Code	aller à une ligne par son numéro
Ctrl+S	éditeur	enregistrer le fichier (sans cela, rien n'est changé sur le disque)
Ctrl+Z	partout	annuler la dernière modification
Ctrl+F5	navigateur	recharger la page en relisant tout . Le navigateur garde une copie des fichiers en mémoire ; F5 seul peut réutiliser l'ancienne. Après avoir modifié le compilateur, c'est toujours Ctrl+F5.
F12	navigateur	ouvre les outils de développement ; l'onglet Console montre les erreurs de la page, avec le fichier et la ligne
AltGr+7	clavier français	l' accent grave seul,  , dont la partie E a besoin (appuyer ensuite sur Espace)

Un **numéro de ligne** désigne la place d'une ligne dans un fichier : « vers la ligne 5432 » veut dire « à peu près la 5432e ligne en partant du haut ». Dans le Bloc-notes, le numéro s'affiche en bas à droite (menu Affichage → Barre d'état).

0.11 La fenêtre de commande

Une **fenêtre de commande** (un « **terminal** ») est une fenêtre où l'on tape des ordres au clavier, au lieu de cliquer. On s'en sert pour lancer les vérifications du projet.

L'ouvrir dans le dossier du projet : dans l'Explorateur de fichiers, ouvre le dossier du projet, fais un **clic droit dans le vide** → « **Ouvrir dans le Terminal** ». (Autre façon : clique dans la barre d'adresse de l'Explorateur, tape `cmd`, puis Entrée.)

Taper un ordre, puis Entrée. Les trois dont ce document se sert :

<code>node</code> <code>outils/gb3.mjs</code> <code>essai.cpp</code>	compile le fichier <code>essai.cpp</code> et dit s'il est accepté (ou le message d'erreur). <code>node</code> est le programme qui fait tourner du JavaScript en dehors du navigateur.
<code>npm run</code> <code>verifier</code>	lance tous les contrôles du projet : chaque leçon est compilée et jouée. Chaque ligne dit <code>OK</code> ou <code>NON</code> ; à la fin, « tout est vert » si rien n'est cassé.
<code>npm run pdf-</code> <code>include</code>	refait ce document, à jour.

Node.js doit être installé : c'est le cas si l'atelier démarre avec `lancer.bat` (il propose de l'installer sinon).

0.12 git : garder une trace, revenir en arrière

git est un programme qui photographie le projet à chaque étape importante. Si une modification casse tout, on revient à la dernière photo. Le dossier du projet est déjà suivi par git.

Les commandes, à taper dans la fenêtre de commande (section 0.11) :

<code>git status</code>	la liste des fichiers modifiés depuis la dernière photo.
<code>git diff</code>	ce qui a changé, ligne par ligne : <code>-</code> devant une ligne enlevée, <code>+</code> devant une ligne ajoutée. (Touche <code>q</code> pour sortir.)
<code>git add -A</code>	prendre la photo , avec une phrase qui dit ce qu'elle contient.
<code>git commit -m "ajout de pile()"</code>	
<code>git checkout -- compilateur/emetteur.js</code>	annuler toutes les modifications de ce fichier depuis la dernière photo. Attention : ce qui n'a pas été photographié est perdu.

La bonne habitude

Une photo (`git commit`) **avant** de toucher au compilateur, et une autre **après**, quand `npm run verifier` est vert. Si quelque chose casse entre les deux, `git checkout -- ...` remet le fichier comme avant.

Les deux sortes d'#include

• **débutant** aucune connaissance demandée en dehors de ce qui est cité

À lire avant : 0.2 Un fichier, un dossier, une extension · 0.3 · 0.6 · 0.9

Une ligne `#include` veut toujours dire la même chose : « ce programme a besoin de quelque chose qui n'est pas écrit ici ». Ce qui change, c'est où ce quelque chose se trouve. Les signes qui entourent le nom le disent :

	Des guillemets <code>"..."</code>	Des chevrons <code><...></code>
Exemple	<code>#include "outils.cpp"</code>	<code>#include <bande></code>
Ce que c'est	un fichier à toi , un onglet à côté de <code>principal.cpp</code>	une fonction de la console , connue du compilateur
Qui l'a écrit	toi	le projet (ou toi, si tu l'y as ajoutée : partie E)
Ce que fait la ligne	elle est remplacée par tout le texte du fichier, mot pour mot	elle autorise la fonction ; elle reste dans le texte
Le nom	le nom de l'onglet, extension comprise	le nom de la fonction, exactement comme on l'appelle
Sans la ligne	les fonctions du fichier sont inconnues	refus : « il faut <code>#include <bande></code> »
Où on la modifie	dans l'atelier, sans rien installer	dans les fichiers du compilateur (un éditeur de texte)

Les guillemets : « colle ici mon fichier »

C'est une affaire de texte, rien de plus. Avant même de lire le programme, la console cherche les lignes `#include "..."` et remplace chacune par le contenu du fichier nommé. Le compilateur ne voit donc **qu'un seul long texte** : il ne sait même pas qu'il y avait deux onglets.

Conséquence : un programme rangé en quatre fichiers donne **exactement la même cartouche**, à l'octet près, que le même programme écrit d'un seul bloc. Ranger ne coûte rien.

C'est ce que fait le vrai C++. Le « préprocesseur » colle les fichiers avant la compilation. L'atelier fait pareil, avec une simplification : un fichier versé deux fois ne l'est qu'une fois. Pas besoin des « gardes d'inclusion » (`#ifndef ... #define ... #endif`) qu'on voit dans les vrais projets.

```
// principal.cpp
#include <ALPHABET>
#include "outils.cpp"
int main() { bande(...); }

// outils.cpp
#include <poser>
void bande(...) { ... }
void pile(...) { ... }
```

colle →

Ce que le compilateur voit vraiment, après collage :

```
#include <ALPHABET>
#include <poser>
void bande(...) { ... }
void pile(...) { ... }
int main() { bande(...); }
```

← venu d'outils.cpp
← venu d'outils.cpp
← venu d'outils.cpp

Les chevrons : « je me sers de cette fonction de la console »

La console a des dizaines de fonctions toutes prêtes : `texte()`, `poser()`, `note()`, `sprite()`... Chacune prend de la **place** dans la cartouche (son code, ses petites routines, ses tables). Une cartouche ne fait que 32 768 octets.

La règle de l'atelier : aucune n'est là d'office. On écrit `#include <texte>` pour dire « je me sers de `texte()` ». Sans la ligne, le compilateur refuse l'appel et écrit la ligne qui manque. Avec elle, il grave la fonction — **mais seulement si le programme l'appelle vraiment** : une ligne de trop ne coûte rien (la partie G le mesure).

Ce n'est pas un fichier. `#include <bande>` ne colle aucun texte dans ton programme : la ligne reste là, et c'est le compilateur qui la lit comme une autorisation.

Seules quelques fonctions sont natives, sans `#include` : `image()`, `images()`, `retard()`, `ms()` et `secondes()` — la boucle du jeu et ses horloges, qui ne coûtent presque rien.

Le piège : des chevrons autour d'un nom que la console ne connaît pas

Si le nom entre chevrons n'est **pas** une fonction de la console (une faute de frappe, une majuscule, une fonction pas encore ajoutée), le compilateur le cherche comme un **fichier** — et répond « fichier introuvable ». La partie F montre ce message.

Ce qui se passe quand on compile

●● **intermédiaire** il faut avoir lu ce qui est cité, et avoir essayé

À lire avant : 0.3 Un programme, un compilateur, une cartouche · partie A

Compiler, c'est transformer ton texte C++ en nombres que le processeur de la Game Boy exécute (section 0.3). L'atelier le fait en **cinq passes**, toujours dans le même ordre. Cette partie les raconte deux fois : d'abord **en images, sans code** (B.1) ; puis **avec le vrai code du compilateur**, expliqué morceau par morceau (B.2), pour ceux qui veulent aller plus loin.

B.1 — En images, sans code

Imagine que ton programme est une recette, et le compilateur une équipe de cuisine. Cinq personnes s'en occupent, l'une après l'autre :

1. **Le colleur.** Il lit la recette ligne par ligne. Quand il voit `#include "outils.cpp"`, il va chercher la page `outils.cpp` dans le classeur, et la **colle** à la place de la ligne. Quand il voit `#include <bande>`, il ne colle rien : ce n'est pas une page du classeur, c'est un ustensile de la cuisine. Il laisse la ligne.
2. **Le lecteur.** Il lit la recette collée et la découpe en morceaux qu'il comprend : « ici une fonction nommée `main` », « là un appel à `bande` avec quatre nombres », « là une demande d'ustensile : `bande` ».
3. **Le contrôleur.** Il fait la liste des ustensiles que la recette **utilise** (les fonctions de la console appelées), et la compare à ceux qu'elle **demande** (les `#include <...>`). S'il en manque un : il arrête tout, et écrit « il faut `#include <bande>` ». Si la recette a son propre ustensile du même nom (sa propre fonction `bande`), il ne réclame rien : c'est le sien.
4. **Le magasinier.** Pour chaque ustensile demandé **et vraiment utilisé**, il va chercher son mode d'emploi dans la réserve du projet, et l'ajoute au début de la recette. Pour `bande()`, ce mode d'emploi est écrit en C++, exactement comme si tu l'avais écrit toi-même. Un ustensile demandé mais jamais utilisé reste dans la réserve : il ne coûte rien.
5. **Le traducteur.** Il traduit toute la recette (la tienne, plus les modes d'emploi ajoutés) dans la langue du processeur : des nombres. Il les range dans la cartouche, avec les dessins. L'émulateur peut la lancer.

À retenir

Les **guillemets** sont l'affaire du **colleur** (étape 1) : c'est du texte collé, rien de plus.

Les **chevrons** sont l'affaire du **contrôleur** et du **magasinier** (étapes 3 et 4) : une autorisation, puis l'ajout du code de la fonction, seulement si elle sert.

Une fonction de la console écrite en C++ est ajoutée **exactement comme si tu l'avais écrite**. C'est pour cela qu'en ajouter une est à ta portée : il suffit de la ranger dans la réserve (partie E).

B.2 — Avec le vrai code

Cette section montre le vrai code du compilateur, lu dans ses fichiers pendant la fabrication de ce document. Il est en JavaScript : la partie D explique ce qu'il faut pour le lire. Tu peux la sauter sans rien perdre pour la suite.

Étape 1 — Le colleur : reconnaître une ligne `#include`

Dans `compilateur/inclusion.js`, une seule ligne décide si une ligne du programme est un `#include`. C'est un **motif** (on dit une « expression régulière ») : une sorte de pochoir que le compilateur pose sur chaque ligne pour voir si elle a la bonne forme.

`compilateur/inclusion.js`

```
const MOTIF = /^[ \t]*#\s*include\s*(?:\"([^\"]+)\"|<([>]+)>)[ \t]*(?:\\/\\.*)?$/
```

Le pochoir, morceau par morceau (le motif est entre les deux `/`) :

Morceau	Ce qu'il veut dire	Dans <code>#include <bande></code> <code>// ...</code>
<code>^</code>	le début de la ligne	avant le <code>#</code>
<code>[\t]*</code>	des espaces ou des tabulations, autant qu'on veut (même aucun)	aucun
<code>#</code>	le signe dièse, tel quel	<code>#</code>
<code>\s*</code>	des espaces facultatifs	aucun
<code>include</code>	le mot <code>include</code> , tel quel	<code>include</code>
<code>\s*</code>	des espaces facultatifs	un espace
<code>"([^\"]+)"</code>	soit un nom entre guillemets (<code>[^\"]+</code> : un ou plusieurs signes qui ne sont pas des guillemets) — c'est un fichier	—
<code> </code>	« ou bien »	
<code><([>]+)></code>	soit un nom entre chevrons	<code><bande></code> → le nom <code>bande</code>
<code>[\t]*</code>	des espaces facultatifs	les trois espaces
<code>(?:\\/\\.*)?</code>	un commentaire <code>//</code> facultatif, jusqu'au bout	<code>// ...</code>
<code>\$</code>	la fin de la ligne	

Ce qui se passe ensuite : si le nom est entre guillemets, le colleur remplace la ligne par le fichier. S'il est entre chevrons **et** qu'il figure dans la liste `BIBLIOTHEQUES` (partie J), la ligne est laissée telle quelle. S'il est entre

chevrons mais **absent** de la liste, il est traité comme un nom de fichier — d'où le message « fichier introuvable » de la partie F.

Étape 3 — Le contrôleur

Dans `compilateur/emetteur.js`, la fonction `inclusionsEmployees` parcourt tout l'arbre du programme. Pour chaque **appel** dont le nom est dans `BIBLIOTHEQUES` — et qui n'est pas une fonction écrite par le programme lui-même —, elle note le nom et sa ligne. Ensuite, ce qui est noté sans être inclus est refusé, avant toute traduction.

Étape 4 — Le magasinier : ajouter les fonctions écrites en C++

Voici, lue dans `compilateur/emetteur.js`, la boucle qui ajoute les fonctions de la console écrites en C++ :

`compilateur/emetteur.js`

```
for (const [nom, source] of Object.entries(FONCTIONS_EN_C)) {  
  if (siennes.has(nom) || !appelle(programme, nom)) continue // la sienne, ou pas appelée  
  ajoutees.push(...analyser(source))  
}
```

Ligne par ligne :

ligne 1	<code>for (const [nom, source] of Object.entries(FONCTIONS_EN_C))</code> : pour chaque élément de l'objet <code>FONCTIONS_EN_C</code> (D.4), prendre son nom (<code>bande</code>) et sa valeur — la source C++ (<code>SOURCE_BANDE</code>).
ligne 2	<code>if (siennes.has(nom) !appelle(programme, nom)) continue</code> : si le programme a écrit sa propre fonction de ce nom, ou (<code> </code>) s'il ne l'appelle pas (<code>!</code>), passer à la suivante (<code>continue</code>) sans rien ajouter.
ligne 3	<code>ajoutees.push(...analyser(source))</code> : sinon, lire la source C++ avec le même lecteur que ton programme (<code>analyser</code>), et ajouter ce qu'il en tire à la liste des fonctions ajoutées.
ligne 4	<code>}</code> : fin de la boucle.

Un peu plus bas, chaque fonction ajoutée reçoit une marque, `deLaConsole` : c'est elle qui lui évite d'avoir à écrire ses propres `#include` (`bande()` appelle `poser()` sans `#include <poser>`).

Le chapitre du parcours, étape par étape

•• **intermédiaire** il faut avoir lu ce qui est cité, et avoir essayé

À lire avant : 0.5 L'écran : des cases, des tuiles, la police · 0.6 · 0.7 · 0.8 · partie A

Dans l'atelier, ce chapitre est le dernier du parcours (bouton  **APPRENDRE**), sous le nom « Tes propres #include : ajouter une fonction ». Dans la page du tutoriel, il commence à la leçon 144 : `tuto.html#lecon-619`.

Chaque étape reprend la précédente et ne change qu'une chose. Les voici toutes, avec leur programme complet (et leurs fichiers voisins), et l'écran qu'elles donnent vraiment.

leçon 144 du parcours

144. Une fonction à toi : bande()

Avant de parler d'#include, on écrit une fonction à soi : bande() pose la même tuile plusieurs fois, de gauche à droite.

Ce chapitre répond à une question : « comment ajouter moi-même un `#include` ? » Il y a deux sortes d'#include, et on va les faire toutes les deux, avec **une seule fonction**, du début à la fin : `bande()`.

On part de ce qu'on sait déjà faire (chapitre « Les fonctions : nommer un geste ») : écrire une fonction dans son programme. `bande(colonne, ligne, tuile, longueur)` pose la tuile `longueur` fois, une case plus à droite à chaque fois.

Lis la fonction ligne par ligne : `void` dit qu'elle ne rend rien ; entre les parenthèses, ses **quatre paramètres**, quatre cases de mémoire remplies par l'appel. La boucle `for` compte `i` de 0 jusqu'à `longueur - 1`, et pose la tuile en `colonne + i`.

Déroulé de `bande(2, 5, ALPHABET[0], 10)` : $i = 0 \rightarrow$ case (2, 5) ; $i = 1 \rightarrow$ case (3, 5) ; ... ; $i = 9 \rightarrow$ case (11, 5). Puis $i = 10$: `10 < 10` est faux, la boucle s'arrête. Dix A, des colonnes 2 à 11.

Pour l'instant, aucun `#include` nouveau : la fonction est **dans** le programme, elle n'a rien à demander à la console. Elle se sert seulement de `poser()`, déjà incluse.

le programme

```
// Une fonction à toi : bande().
// Elle est écrite ICI, dans le programme : pas besoin d'#include pour elle.

#include <poser>      // pose une tuile sur une case du fond (bande() s'en sert)
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A

// bande(colonne, ligne, tuile, longueur) : la même tuile, « longueur » fois,
// de gauche à droite. Exemple : bande(2, 5, ALPHABET[0], 10) → dix A,
// des colonnes 2 à 11, sur la ligne 5.
//
// void          elle ne rend rien : elle agit, c'est tout
// uint8_t ...   ses quatre paramètres : des nombres de 0 à 255
void bande(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t longueur) {
    // for (départ ; condition ; après chaque tour)
    //   i = 0 au départ ; on continue tant que i < longueur ; i++ ajoute 1.
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, tuile); // une case plus à droite à chaque tour
    }
}
```



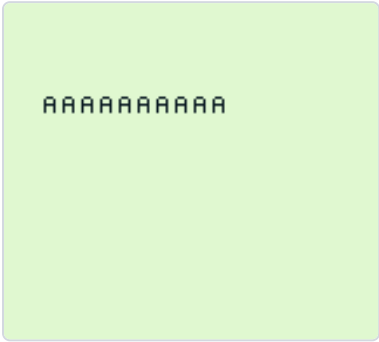
```

    }
}

int main() {
    bande(2, 5, ALPHABET[0], 10); // l'appel : dix A sur la ligne 5

    while (true) { // la boucle du jeu
        image();    // attend l'image suivante (native : pas d'#include)
    }
}

```



AAAAAAAAAAAA

l'écran, dans l'émulateur — 911 octets

CE QU'ON DOIT VOIR

Dix A côte à côte sur la ligne 5, des colonnes 2 à 11.

leçon 145 du parcours

145. La même fonction, trois fois

Une fonction s'écrit une fois et s'appelle autant qu'on veut : trois bandes, trois longueurs, trois lettres.

C'est le programme d'avant, avec deux appels de plus dans `main()`. La fonction, elle, ne change pas d'une lettre.

Chaque appel remplit les paramètres autrement : `bande(2, 7, ALPHABET[1], 6)` met 2 dans `colonne`, 7 dans `ligne`, le B dans `tuile`, 6 dans `longueur`. Six B, des colonnes 2 à 7.

C'est tout l'intérêt d'une fonction : le geste (« poser une rangée ») est écrit **une seule fois**. C'est aussi ce qui va nous donner envie de la **ranger à part** : une fonction aussi utile, on voudrait la réemployer dans d'autres programmes, sans la recopier.

Essaie : ajoute `bande(0, 11, ALPHABET[3], 20);` — une ligne entière de D, de la colonne 0 à la 19.

le programme

```

// La même fonction, trois fois : seuls les appels de main() changent.

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // ALPHABET[0] = A, [1] = B, [2] = C...

// bande(colonne, ligne, tuile, longueur) : comme à l'étape d'avant.
void bande(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, tuile);
    }
}

int main() {
    bande(2, 5, ALPHABET[0], 10); // dix A : colonnes 2 à 11, ligne 5
    bande(2, 7, ALPHABET[1], 6);  // NOUVEAU : six B : colonnes 2 à 7, ligne 7
}

```

```

bande(2, 9, ALPHABET[2], 3);    // NOUVEAU : trois C : colonnes 2 à 4,  ligne 9

while (true) {
    image();
}
}

```

```

AAAAAAAAAAAA
BBBBBBB
CCC

```

l'écran, dans l'émulateur — 949 octets

CE QU'ON DOIT VOIR

Trois rangées : dix A, six B, trois C, toutes calées à gauche sur la colonne 2.

leçon 146 du parcours

146. Ranger sa fonction dans un fichier voisin : `#include "outils.cpp"`

La première sorte d'`#include`, avec des guillemets : `bande()` déménage dans l'onglet `outils.cpp`, et `principal.cpp` le verse chez lui.

Ce qui est nouveau ici : la fonction a déménagé. Elle n'est plus dans `principal.cpp` : elle est dans un **second onglet**, `outils.cpp`. Dans l'atelier, c'est le bouton « + fichier », au-dessus de l'éditeur, qui crée un onglet comme celui-là.

`#include "outils.cpp"` veut dire « verse ici tout le texte d'`outils.cpp` ». Avant de compiler, la console remplace cette ligne par le contenu du fichier, mot pour mot. Le compilateur voit donc **exactement** le programme de l'étape d'avant : même cartouche, même écran.

Les guillemets `"..."` désignent un fichier **À TOI**, écrit à côté du programme. Retiens-le bien, car l'étape 15.6 montrera l'autre sorte, avec des chevrons `<...>`, qui désigne une fonction **de la console**.

Où placer la ligne : l'habitude est de mettre tous les `#include` en haut, ensemble. Pour des **fonctions**, l'ordre ne compte pas : le compilateur relève toutes les fonctions avant de les traduire, et `main()` peut appeler `bande()` même si son texte est collé plus bas. Pour des **variables globales** ou des **dessins**, si : ils doivent être versés **avant** la fonction qui s'en sert.

Pourquoi l'extension `.cpp` et pas `.h` : les deux marchent ici. Par habitude, un `.h` (« header ») contient des déclarations (des variables, des noms) ; un `.cpp` contient du code, des fonctions. `outils.cpp` contient une fonction.

`outils.cpp`

```

// outils.cpp : MES fonctions, rangées à part.
// Ce fichier ne fait rien tout seul : principal.cpp le verse chez lui
// avec #include "outils.cpp" (des GUILLEMETS : c'est un fichier à moi).

// bande(colonne, ligne, tuile, longueur) : la même tuile, « longueur » fois,
// de gauche à droite. Exemple : bande(2, 5, ALPHABET[0], 10) → dix A.
void bande(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, tuile);    // poser() : incluse par principal.cpp
    }
}

```

```
}  
}
```

principal.cpp

```
// Le changement : bande() n'est plus écrite ici, mais dans outils.cpp.  
//  
// #include "outils.cpp"  
// |  
// |      +-- le fichier à verser : l'onglet « outils.cpp », à côté  
// +----- « verse ici » : avant la compilation, cette ligne est  
//           remplacée par tout le texte d'outils.cpp. Tout se passe  
//           comme si bande() était écrite ici, comme à l'étape d'avant.  
  
#include <poser>      // pose une tuile (c'est bande() qui s'en sert)  
#include <ALPHABET>   // les lettres de la police  
#include "outils.cpp" // NOUVEAU : mes fonctions à moi, versées ici  
  
int main() {  
    bande(2, 5, ALPHABET[0], 10); // bande() vient d'outils.cpp  
    bande(2, 7, ALPHABET[1], 6);  
    bande(2, 9, ALPHABET[2], 3);  
  
    while (true) {  
        image();  
    }  
}
```

```
AAAAAAAAAA  
BBBBBB  
CCC
```

l'écran, dans l'émulateur — 949 octets

CE QU'ON DOIT VOIR

Exactement l'écran d'avant : dix A, six B, trois C. Mais le programme est rangé en deux onglets.

leçon 147 du parcours

147. Le fichier voisin écrit ses propres #include <...>

outils.cpp se sert de poser() : c'est donc lui qui écrit #include <poser>. Le fichier se suffit à lui-même.

Le seul changement : la ligne #include <poser> a changé d'onglet. Elle était dans principal.cpp ; elle est maintenant en haut d'outils.cpp.

Pourquoi c'est mieux : c'est bande() qui se sert de poser(), pas main(). En écrivant la ligne **dans le fichier qui en a besoin**, outils.cpp se suffit à lui-même : un autre programme qui écrit #include "outils.cpp" n'a rien d'autre à penser.

Et si deux fichiers écrivent la même ligne ? Si principal.cpp gardait aussi son #include <poser>, ce ne serait pas une erreur : une fonction de la console n'est gravée **qu'une fois**, et une ligne de trop ne coûte rien.

Le texte versé, en entier : `#include <ALPHABET>`, puis tout `utils.cpp` (son `#include <poser>` et `bande()`), puis `main()`. Un `#include <...>` vaut pour **tout** le programme assemblé, où qu'il soit écrit : celui d'`utils.cpp` autorise `poser()` partout.

`utils.cpp`

```
// utils.cpp : MES fonctions, rangées à part.
// Il écrit LUI-MÊME ce dont il a besoin : il se suffit à lui-même.

#include <poser>    // NOUVEAU ICI : bande() se sert de poser()

// bande(colonne, ligne, tuile, longueur) : la même tuile, « longueur » fois,
// de gauche à droite. Exemple : bande(2, 5, ALPHABET[0], 10) → dix A.
void bande(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, tuile);
    }
}
```

`principal.cpp`

```
// Le changement : « #include <poser> » est parti dans utils.cpp,
// le fichier qui s'en sert. main() n'appelle pas poser() lui-même.

#include <ALPHABET>    // les lettres : c'est main() qui s'en sert
#include "utils.cpp"    // mes fonctions (et leurs #include à elles)

int main() {
    bande(2, 5, ALPHABET[0], 10);
    bande(2, 7, ALPHABET[1], 6);
    bande(2, 9, ALPHABET[2], 3);

    while (true) {
        image();
    }
}
```

```
AAAAAAAAAA
BBBBBB
CCC
```

l'écran, dans l'émulateur — 949 octets

CE QU'ON DOIT VOIR

Toujours le même écran : dix A, six B, trois C.

leçon 148 du parcours

148. Une deuxième fonction dans `utils.cpp` : `pile()`

Un fichier d'utils grandit : `pile()` fait comme `bande()`, mais vers le bas. `principal.cpp` n'a rien à ajouter pour s'en servir.

Ce qui est nouveau ici : `pile(colonne, ligne, tuile, hauteur)`, une deuxième fonction dans `utils.cpp`. C'est `bande()` tournée d'un quart de tour : la tuile est posée **vers le bas**, en `ligne + i`.

Déroulé de `pile(15, 4, ALPHABET[3], 6)` : $i = 0 \rightarrow (15, 4)$; $i = 1 \rightarrow (15, 5)$; ... ; $i = 5 \rightarrow (15, 9)$. Six D, l'un sous l'autre.

Aucune ligne ajoutée dans principal.cpp pour l'avoir : le `#include "outils.cpp"` verse **tout** le fichier, donc toutes ses fonctions. Un fichier d'outils, c'est cela : une boîte où l'on range ses fonctions, et qu'on ouvre d'une seule ligne.

Essaie : écris une troisième fonction dans `outils.cpp`, par exemple `void carreDe(colonne, ligne, tuile)` qui appelle `bande()` deux fois.

`outils.cpp`

```
// outils.cpp : MES fonctions, rangées à part.

#include <poser> // bande() et pile() se servent de poser()

// bande(colonne, ligne, tuile, longueur) : vers la DROITE.
// Exemple : bande(2, 5, ALPHABET[0], 10) → dix A, colonnes 2 à 11.
void bande(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, tuile); // la COLONNE avance
    }
}

// NOUVEAU : pile(colonne, ligne, tuile, hauteur) : vers le BAS.
// Exemple : pile(15, 4, ALPHABET[3], 6) → six D, lignes 4 à 9.
void pile(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t hauteur) {
    for (uint8_t i = 0; i < hauteur; i++) {
        poser(colonne, ligne + i, tuile); // c'est la LIGNE qui avance
    }
}
```

`principal.cpp`

```
// Le changement : main() appelle aussi pile(), la nouvelle fonction
// d'outils.cpp. Rien à ajouter en haut : outils.cpp est déjà versé.

#include <ALPHABET>
#include "outils.cpp" // bande() ET pile()

int main() {
    bande(2, 5, ALPHABET[0], 10);
    bande(2, 7, ALPHABET[1], 6);
    bande(2, 9, ALPHABET[2], 3);
    pile(15, 4, ALPHABET[3], 6); // NOUVEAU : six D, de la ligne 4 à la 9

    while (true) {
        image();
    }
}
```

```
AAAAAAAAAA
BBBBBB
CCC
D
D
D
D
D
D
D
```

CE QU'ON DOIT VOIR

Les trois bandes d'avant, et une colonne de six D à droite (colonne 15, lignes 4 à 9).

148.1. La fonction `bande()` — la même tuile, plusieurs fois

#include <bande> : `bande(colonne, ligne, tuile, longueur)` pose la même tuile « longueur » fois, de gauche à droite.

Ce qu'elle fait : `bande(2, 5, ALPHABET[0], 10)` pose **dix A** sur la ligne 5, des colonnes 2 à 11 — comme dix `poser()` d'affilée, une case plus à droite à chaque fois.

La ligne à écrire : `#include <bande>`.

Ses arguments : la colonne et la ligne de la première case, la tuile, et combien de fois la poser.

D'où elle vient : c'est la fonction que le chapitre « Tes propres #include » écrit d'abord dans le programme, puis range dans un fichier voisin, et fait enfin entrer dans la console. Elle est écrite en C, comme une fonction de l'élève (`compilateur/emetteur.js`, `SOURCE_BANDE`).

Ce qu'elle coûte : sa boucle, et `poser()` qu'elle appelle — gravées seulement si le programme appelle `bande()`.

Essaie : `bande(0, 17, ALPHABET[1], 20);` — une ligne de B tout en bas.

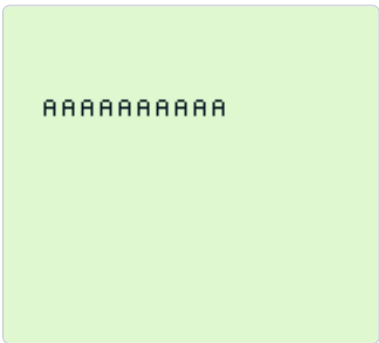
le programme

```
// ---- #include <bande> : la même tuile, plusieurs fois ----
// Dix A sur la ligne 5, des colonnes 2 à 11, en un seul appel.

#include <ALPHABET>    // les lettres : ALPHABET[0] est le A
#include <bande>       // bande() : la même tuile, de gauche à droite

int main() {
    bande(2, 5, ALPHABET[0], 10);    // colonne 2, ligne 5, le A, dix fois

    while (true) {
        image();
    }
}
```



AAAAAAAAAA

CE QU'ON DOIT VOIR

Dix A côte à côte sur la ligne 5.

l'écran, dans l'émulateur — 911 octets

149. bande() devient une fonction de la console : #include <bande>

La seconde sorte d'#include, avec des chevrons : bande() a été ajoutée à la console elle-même. On ne l'écrit plus, on la demande.

Ce qui est nouveau ici : #include <bande>, avec des chevrons. bande() n'est plus dans outils.cpp (il n'y reste que pile()) : elle fait maintenant partie **de la console**, comme poser() ou texte(). On ne l'écrit plus, on la **demande**.

Guillemets ou chevrons, la différence en une phrase : #include "outils.cpp" verse **ton fichier** ; #include <bande> demande **une fonction de la console**, que le compilateur connaît déjà et n'ajoute à la cartouche que si le programme l'appelle.

Sans la ligne, le compilateur refuse : efface #include <bande> et lance. Le message dit : « il faut #include <bande> pour employer bande() ». C'est la règle de toute la console : ce qu'on emploie, on l'inclut par son nom.

Comment bande() est entrée dans la console. Il a fallu toucher **trois fichiers du projet** (pas le programme : le compilateur lui-même). On les ouvre dans un éditeur de texte, à côté d'index.html :

1. compilateur/emetteur.js — le code de la fonction. On y écrit sa source, **en C, exactement comme dans outils.cpp**, dans une constante, SOURCE_BANDE, entre deux accents graves (la touche AltGr + 7) : le texte de la fonction, mot pour mot. Puis on l'inscrit dans le tableau FONCTIONS_EN_C, juste en dessous : bande: SOURCE_BANDE,. Le compilateur ajoute ce texte au programme **seulement** s'il appelle bande().

2. compilateur/inclusion.js — le nom à inclure. Dans le tableau BIBLIOTHEQUES, une ligne : bande: 'pose la même tuile plusieurs fois, de gauche à droite',. C'est elle qui rend #include <bande> valable, et la phrase sert de commentaire quand l'atelier écrit les #include tout seul.

3. aide-fonctions.js — l'aide de l'éditeur. Dans FONCTIONS : bande: { args: ['colonne', 'ligne', 'tuile', 'longueur'], dit: '...' },. L'éditeur propose alors bande quand on tape « ban... », et montre les arguments pendant qu'on les écrit.

Un détail : dans la console, bande() n'écrit pas #include <poser>. Ce que la console ajoute elle-même n'a rien à inclure : le compilateur le sait (c'est le drapeau deLaConsole).

Après avoir changé ces fichiers : recharge la page avec **Ctrl+F5**. Le compilateur est relu, et #include <bande> marche aussitôt.

outils.cpp

```
// outils.cpp : MES fonctions. bande() n'y est plus : elle est
// dans la console, demandée par #include <bande> dans principal.cpp.

#include <poser> // pile() se sert de poser()

// pile(colonne, ligne, tuile, hauteur) : vers le BAS.
// Exemple : pile(15, 4, ALPHABET[3], 6) → six D, lignes 4 à 9.
void pile(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t hauteur) {
    for (uint8_t i = 0; i < hauteur; i++) {
        poser(colonne, ligne + i, tuile);
    }
}
```

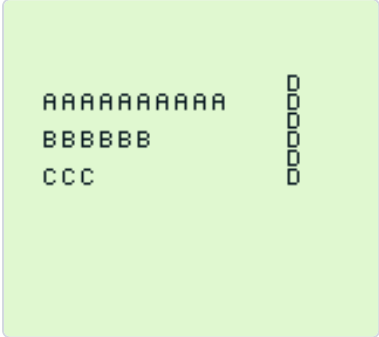
principal.cpp

```
// Le changement : bande() vient de la CONSOLE, plus d'utils.cpp.
//
// #include <bande>      des CHEVRONS : une fonction de la console
// #include "utils.cpp"  des GUILLEMETS : un fichier à moi

#include <ALPHABET>
#include <bande>        // NOUVEAU : pose la même tuile plusieurs fois, de gauche à droite
#include "utils.cpp"    // il ne contient plus que pile()

int main() {
    bande(2, 5, ALPHABET[0], 10); // la bande() de la console
    bande(2, 7, ALPHABET[1], 6);
    bande(2, 9, ALPHABET[2], 3);
    pile(15, 4, ALPHABET[3], 6);  // la pile() d'utils.cpp

    while (true) {
        image();
    }
}
```



```
AAAAAAAAAA
BBBBBBB
CCC
D
D
D
D
D
D
D
D
D
D
```

l'écran, dans l'émulateur — 1010 octets

CE QU'ON DOIT VOIR

Le même écran : trois bandes et une pile. bande() vient maintenant de la console.

leçon 150 du parcours

150. Ta fonction passe avant celle de la console

Si ton programme écrit une fonction du même nom qu'une fonction de la console, c'est la tienne qui compte.

Le seul changement : `utils.cpp` écrit de nouveau une `bande()`, mais **en pointillés** : une case sur deux.

`#include <bande>` est toujours là, dans `principal.cpp`.

Laquelle gagne ? La tienne. Le compilateur ajoute une fonction de la console **seulement si le programme n'en a pas écrit une du même nom**. Ici, il voit ta `bande()` : il n'ajoute pas la sienne, et la ligne `#include <bande>` ne grave rien.

La boucle en pointillés : `i = i + 2` au lieu de `i++`. Déroulé de `bande(2, 5, ALPHABET[0], 10)` : $i = 0 \rightarrow (2, 5)$; $i = 2 \rightarrow (4, 5)$; $i = 4 \rightarrow (6, 5)$; $i = 6 \rightarrow (8, 5)$; $i = 8 \rightarrow (10, 5)$; $i = 10$: `10 < 10` est faux, fin. Cinq A, un trou entre chaque.

À quoi ça sert : à **essayer une autre version** d'une fonction de la console sans toucher au compilateur. Si la tienne te plaît, tu sais maintenant comment la faire entrer dans la console (étape précédente).

Essaie : supprime la `bande()` d'`utils.cpp` : celle de la console revient, et les bandes redeviennent pleines.


```
// outils.cpp : MES fonctions.

#include <poser>

// NOUVEAU : MA bande(), en pointillés. Elle porte le même nom que celle de
// la console : c'est la mienne qui compte, la console n'ajoute pas la sienne.
// Exemple : bande(2, 5, ALPHABET[0], 10) → A aux colonnes 2, 4, 6, 8, 10.
void bande(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i = i + 2) { // i = i + 2 : une case sur deux
        poser(colonne + i, ligne, tuile);
    }
}

// pile(colonne, ligne, tuile, hauteur) : vers le BAS.
void pile(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t hauteur) {
    for (uint8_t i = 0; i < hauteur; i++) {
        poser(colonne, ligne + i, tuile);
    }
}
```

```
// Le changement est dans outils.cpp : il écrit sa propre bande().
// principal.cpp garde « #include <bande> », mais c'est la mienne qui sert.

#include <ALPHABET>
#include <bande> // la bande() de la console... qui ne servira pas ici
#include "outils.cpp" // MA bande() (en pointillés) et pile()

int main() {
    bande(2, 5, ALPHABET[0], 10); // MA bande() : cinq A, un trou entre chaque
    bande(2, 7, ALPHABET[1], 6); // trois B : colonnes 2, 4, 6
    bande(2, 9, ALPHABET[2], 3); // deux C : colonnes 2, 4
    pile(15, 4, ALPHABET[3], 6);

    while (true) {
        image();
    }
}
```

```
A A A A A
B B B
C C
```

```
D
D
D
D
D
D
```

CE QU'ON DOIT VOIR

Les bandes sont en pointillés : A A A A A, B B B, C C. La pile de D ne change pas.

l'écran, dans l'émulateur — 1011 octets

151. À toi : ajouter ta propre fonction de la console, pas à pas

La marche à suivre complète, de l'idée à « #include <ta_fonction> » — et un cadre dessiné avec bande() et pile().

On revient à la `bande()` de la console (`utils.cpp` ne contient plus que `pile()`, comme au 15.6), et on s'en sert pour **un cadre** : deux bandes (le haut et le bas), deux piles (la gauche et la droite). C'est la seule nouveauté du programme.

Déroulé du cadre : `bande(3, 3, ..., 14)` → le haut, colonnes 3 à 16 ; `bande(3, 12, ..., 14)` → le bas ; `pile(3, 4, ..., 8)` → la gauche, lignes 4 à 11 ; `pile(16, 4, ..., 8)` → la droite. Les coins appartiennent aux bandes : les piles commencent une ligne plus bas.

Et maintenant, pour faire entrer `pile()` dans la console à son tour, voici la marche à suivre — la même pour n'importe quelle fonction :

Étape 1 — l'écrire et l'essayer dans ton programme. D'abord dans `principal.cpp`, ou dans un fichier voisin comme `utils.cpp`. Tant qu'elle n'est pas parfaite, elle reste là : c'est plus facile à corriger.

Étape 2 — choisir son nom. Un nom qu'aucune fonction de la console ne porte déjà (la liste est dans `compilateur/inclusion.js`, tableau `BIBLIOTHEQUES`). Le nom du `#include` sera exactement celui de la fonction : `pile` → `#include <pile>`.

Étape 3 — `compilateur/emetteur.js`. Cherche `const FONCTIONS_EN_C`. Juste au-dessus, colle ta fonction dans une constante, `const SOURCE_PILE = ...`, entre deux accents graves (AltGr + 7), comme `SOURCE_BANDE` juste à côté. Dans le tableau, ajoute `pile: SOURCE_PILE`,. Retire son `#include <poser>` : la console n'en a pas besoin.

Étape 4 — `compilateur/inclusion.js`. Dans `BIBLIOTHEQUES`, ajoute une ligne : `pile: 'pose la même tuile plusieurs fois, vers le bas'`,. Sans elle, `#include <pile>` serait refusé : « je ne connais pas cette bibliothèque ».

Étape 5 — `aide-fonctions.js`. Dans `FONCTIONS`, ajoute `pile: { args: ['colonne', 'ligne', 'tuile', 'hauteur'], dit: '...' }`, pour que l'éditeur la propose.

Étape 6 — essayer. Recharge la page (**Ctrl+F5**), enlève `pile()` d'`utils.cpp`, écris `#include <pile>` dans `principal.cpp`, et lance. En ligne de commande : `node utils/gb3.mjs mon-essai.cpp` compile un fichier, et `npm run verifier` vérifie que rien d'autre n'est cassé.

Étape 7 — sa leçon. Dans ce projet, chaque `#include` a son tuto : une leçon dans `tuto/lecons.js` dont le programme commence par `// ---- #include <pile> : ...`. Le parcours la place tout seul juste avant la première étape qui emploie `pile()`. Copie celle de `bande()` et change ce qu'il faut.

Pour aller plus loin : les fonctions qui parlent directement au matériel (l'écran, le son) ne sont pas écrites en C mais en instructions du processeur, dans `compilateur/emetteur.js` (cherche `if (nom === 'cacherPanneau')`). C'est plus difficile : la façon en C suffit pour tout ce qu'on peut écrire avec les fonctions existantes.

`utils.cpp`

```
// utils.cpp : MES fonctions. pile() est la prochaine à faire entrer
// dans la console, en suivant les étapes 1 à 7 de cette leçon.

#include <poser> // pile() se sert de poser()

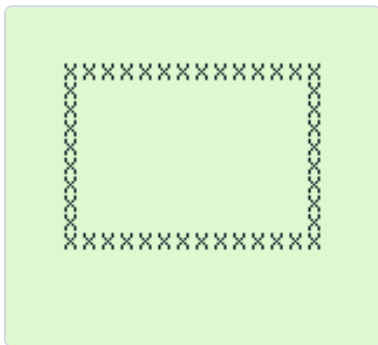
// pile(colonne, ligne, tuile, hauteur) : vers le BAS.
// Exemple : pile(3, 4, ALPHABET[23], 8) → huit X, lignes 4 à 11.
void pile(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t hauteur) {
    for (uint8_t i = 0; i < hauteur; i++) {
        poser(colonne, ligne + i, tuile);
    }
}
```

```
// Un cadre de X : bande() (de la console) pour le haut et le bas,
// pile() (d'outils.cpp) pour les côtés.
//
// colonnes 3 à 16, lignes 3 à 12 :
//
//      XXXXXXXXXXXXXXXX ← bande(3, 3, X, 14) le haut
//      X              X ← pile(3, 4, X, 8) et pile(16, 4, X, 8)
//      X              X les côtés, lignes 4 à 11
//      XXXXXXXXXXXXXXXX ← bande(3, 12, X, 14) le bas

#include <ALPHABET>
#include <bande> // pose la même tuile plusieurs fois, de gauche à droite
#include "outils.cpp" // pile()

int main() {
    bande(3, 3, ALPHABET[23], 14); // le haut (ALPHABET[23] : le X, 24e lettre)
    bande(3, 12, ALPHABET[23], 14); // le bas
    pile(3, 4, ALPHABET[23], 8); // le côté gauche : lignes 4 à 11
    pile(16, 4, ALPHABET[23], 8); // le côté droit

    while (true) {
        image();
    }
}
```



l'écran, dans l'émulateur — 1010 octets

CE QU'ON DOIT VOIR

Un cadre de X au milieu de l'écran : colonnes 3 à 16, lignes 3 à 12, vide à l'intérieur.

152. Un raccourci à toi : écrire poserDevant() soi-même

poserDevant() de la console tient en deux lignes : on l'écrit soi-même, dans outils.cpp. Et l'on découvre pourquoi celle de la console est plus légère.

Une fonction « raccourci », c'est juste un nom donné à des gestes qu'on sait déjà faire. poserDevant() (le 110.10) pose une tuile, puis la teint | DEVANT. Ici, on l'écrit **soi-même**, dans outils.cpp, comme bande() et pile().

Lis-la : quatre paramètres, deux lignes. poser(colonne, ligne, tuile) pose la tuile ; teindre(colonne, ligne, palette | DEVANT) choisit sa palette et la fait passer devant les lutins.

Elle porte le même nom que celle de la console : c'est donc la tienne qui sert (le 15.7). Le programme n'écrit pas #include <poserDevant> : il n'en a pas besoin. Il inclut poser et teindre, dont TA fonction se sert.

La différence de poids : ta version est une vraie fonction. À chaque appel, le programme range quatre nombres, saute dans la fonction, et poser() et teindre() ne savent plus rien d'avance (la case, la tuile) : elles

calculent tout pendant le jeu. Celle de la console, elle, n'est **pas une vraie fonction** : le compilateur **recopie ses deux lignes à la place de l'appel**, avec les vrais nombres. Le résultat est le même à l'écran ; la cartouche de la console est plus petite.

Alors, la tienne ne sert à rien ? Si : elle est à toi, tu peux la changer. Une version qui pose **deux** cases d'un coup (un buisson de 2 de large), une autre qui choisit toute seule la palette... La console ne fera jamais ce que ton jeu a en tête.

À toi : ajoute dans `outils.cpp` un `buissonDevant(colonne, ligne)` qui appelle quatre fois `poserDevant()` pour un buisson de 2×2 cases.

`outils.cpp`

```
// outils.cpp : MES fonctions.

#include <poser>      // poserDevant() se sert de poser()...
#include <teindre>    // ... et de teindre()

// poserDevant(colonne, ligne, tuile, palette) : MA version, en deux lignes.
// Même nom que celle de la console : c'est la mienne qui sert.
//  poserDevant(12, 8, BUISSON, 2) → la touffe en (12, 8), palette 2, devant.
void poserDevant(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t palette) {
    poser(colonne, ligne, tuile);           // 1. la tuile, sur sa case
    teindre(colonne, ligne, palette | DEVANT); // 2. sa palette, ET devant les lutins
}
```

`principal.cpp`

```
// Le buisson DEVANT, avec MA poserDevant(), écrite dans outils.cpp.
// Pas de « #include <poserDevant> » : ma fonction vient d'outils.cpp.

#include <Tuile>          // un dessin de 8 × 8 pixels
#include <couleurFond>    // choisit une couleur d'une palette du fond
#include <couleurLutin>   // choisit une couleur d'une palette des lutins
#include <sprite>         // place un lutin de 8 × 8 au pixel près
#include <teindreLutin>   // met un lutin dans une palette
#include "outils.cpp"     // MA poserDevant()

Tuile BUISSON = {        // Une touffe de feuilles : AUCUN 0, elle est toute pleine.
    "11222211",          // (des coins en 0 laisseraient un trou là où deux
    "12233221",          // touffes se touchent : on y verrait le héros)
    "22333322",
    "23333332",
    "23333332",
    "22333322",
    "12233221",
    "11222211",
};

Tuile HEROS = {          // 0 transparent, 1 clair, 2 moyen, 3 contour.
    "00333300",
    "03222230",
    "32122123",
    "32222223",
    "03222230",
    "00333300",
    "03300330",
    "33000333",
};

int main() {
    couleurFond(2, 1, 16, 30, 8); // palette 2 : les verts du buisson
    couleurFond(2, 2, 6, 22, 4);
    couleurFond(2, 3, 0, 10, 2);
```

```

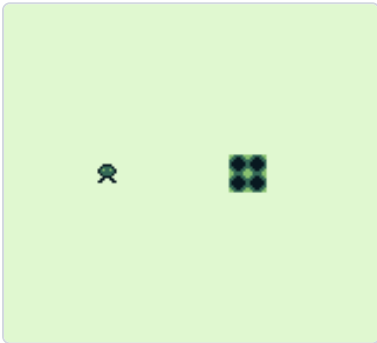
couleurLutin(0, 2, 31, 4, 2); // le héros : rouge
couleurLutin(0, 3, 10, 0, 0);

// Un buisson de 2 x 2 cases, colonnes 12-13, lignes 8-9 : quatre appels.
poserDevant(12, 8, BUISSON, 2);
poserDevant(13, 8, BUISSON, 2);
poserDevant(12, 9, BUISSON, 2);
poserDevant(13, 9, BUISSON, 2);

sprite(0, 96, 68, HEROS);      // le héros, dans le buisson : caché
teindreLutin(0, 0);
sprite(1, 40, 68, HEROS);      // un autre, à découvert : visible
teindreLutin(1, 0);

while (true) {
    image();
}
}

```



l'écran, dans l'émulateur — 1187 octets

CE QU'ON DOIT VOIR

Un buisson vert, et un héros rouge à gauche. Le second héros, dans le buisson, est caché.

Le JavaScript qu'il faut pour la suite

●● **intermédiaire** il faut avoir lu ce qui est cité, et avoir essayé

À lire avant : 0.2 Un fichier, un dossier, une extension · 0.10

Pourquoi du JavaScript ? Tes programmes Game Boy sont en C++. Mais l'**atelier** et son **compilateur**, eux, sont écrits en **JavaScript** : c'est le langage que les navigateurs savent faire tourner. Pour ajouter une fonction à la console (partie E), on modifie trois fichiers JavaScript.

Bonne nouvelle : il n'y a pas besoin d'apprendre le JavaScript. On ne touche qu'à **trois endroits**, toujours de la même façon : on recopie une ligne qui existe déjà, et on change le nom. Cette partie explique juste assez pour lire ces lignes, et ne pas les casser.

D.1 — Les commentaires

Comme en C++ : `//` commente jusqu'au bout de la ligne. En plus, `/* ... */` commente tout ce qui est entre les deux, même sur plusieurs lignes. Les fichiers du compilateur en sont pleins : ils expliquent le code.

```
// un commentaire d'une ligne
/*
 * un commentaire de plusieurs lignes :
 * les étoiles au début de chaque ligne sont juste décoratives
 */
```

D.2 — Donner un nom à une valeur : `const`

```
const SOURCE_BANDE = 'un texte'
|                   |
|                   |└─ la valeur
|                   └─ « reçoit »
|                   └─ le nom choisi
└─ « je crée un nom qui ne changera plus »
```

`const` veut dire « constante » : un nom donné une fois pour toutes à une valeur. Ensuite, écrire `SOURCE_BANDE` ailleurs dans le fichier, c'est comme écrire la valeur elle-même. En JavaScript, le point-virgule au bout de la ligne est facultatif : le projet ne le met pas.

D.3 — Les textes : trois façons de les entourer

Écrit	Ce que c'est	À savoir
'pose une tuile'	un texte entre apostrophes droites	sur une seule ligne. Une apostrophe droite <code>'</code> à l'intérieur le fermerait trop tôt : utilise l'apostrophe typographique <code>'</code> (« l'écran »).
"pose une tuile"	un texte entre guillemets droits	la même chose, avec <code>"</code> .
`...`	un texte entre accents graves (AltGr+7)	peut tenir sur plusieurs lignes : c'est ce qu'il faut pour y mettre toute une fonction C++.

C'est l'astuce de la partie E : la source C++ d'une fonction de la console est simplement **un texte**, rangé entre deux accents graves, dans une constante. Le compilateur lira ce texte comme si tu l'avais écrit dans ton programme.

```
const SOURCE_PILE = `
void pile(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t hauteur) {
    ...
}
`
// ↑ tout ce qui est entre les deux accents graves est UN texte, sur 5 lignes
```

D.4 — Les objets : une liste de « nom : valeur »

Un **objet** est une liste où chaque valeur a un nom. Il s'écrit entre accolades, chaque élément sous la forme `nom: valeur`, **suivi d'une virgule** :

```
export const BIBLIOTHEQUES = {
  texte: 'écrit un texte à l\'écran',
  poser: 'pose une tuile sur une case du fond',
  bande: 'pose la même tuile plusieurs fois, de gauche à droite',
}
```

<code>export</code>	« les autres fichiers ont le droit de s'en servir ». À laisser tel quel.
<code>{ ... }</code>	le début et la fin de l'objet. Tout ce qu'on ajoute va entre les deux.
<code>bande:</code>	le nom de l'élément, suivi de deux-points. Ici, c'est le nom de la fonction de la console.
<code>'pose la même...'</code>	la valeur : ici, un texte (D.3).
<code>,</code>	la virgule sépare un élément du suivant. Le projet en met une même après le dernier : ainsi, on peut toujours ajouter une ligne en dessous sans rien toucher d'autre.

Ajouter un élément, c'est ajouter une ligne `nom: valeur,` entre les accolades. L'ordre n'a pas d'importance pour le compilateur ; le projet range les noms par sujet.

D.5 — Un objet dans un objet, et les tableaux

Une valeur peut être elle-même un objet, ou un **tableau** — une liste sans noms, entre crochets `[ ]`, les éléments séparés par des virgules. C'est le cas de l'aide de l'éditeur :

```
bande: { args: ['colonne', 'ligne', 'tuile', 'longueur'], dit: 'pose la même tuile...' },
      |   |         un tableau de 4 textes          |   | un texte
      |   |_____|                                |   |_____|
      |   | nom : args                            |   | nom : dit
      |   | début de l'objet « fiche de bande »    |   | fin de l'objet _____|
      |_____| nom de l'élément, dans le grand objet FONCTIONS
```

Ici : l'élément `bande` du grand objet `FONCTIONS` est un petit objet à deux éléments, `args` (la liste des noms d'arguments) et `dit` (la phrase d'aide). La virgule tout au bout sépare `bande` de l'élément suivant du grand objet.

D.6 — Ce que tu n’as pas besoin de comprendre

Tout le reste. `compilateur/emetteur.js` fait plus de 6 000 lignes : la traduction en instructions du processeur, les optimisations, les registres... Pour ajouter une fonction écrite en C, **tu n’y touches pas**. Tu ajoutes une constante (D.2, D.3) et une ligne dans un objet (D.4). Rien d’autre.

Si tu vois des mots inconnus autour (`function`, `return`, `=>`, `export`, `import`), laisse-les : ils font partie du compilateur, pas de ce que tu ajoutes.

D.7 — Exercice : trouve les trois erreurs

```
export const BIBLIOTHEQUES = {  
  bande: 'pose la même tuile plusieurs fois'  
  pile: 'pose la même tuile vers le bas',  
  cadre: 'dessine un cadre autour de l'écran',  
}  
const SOURCE_PILE = `  
void pile(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t hauteur) { ... }`
```

Les réponses

1. Ligne 2 : il manque la **virgule** après `'pose la même tuile plusieurs fois'`. Le compilateur ne sait plus où finit l’élément `bande`.
2. Ligne 4 : `l'écran` contient une **apostrophe droite**, qui ferme le texte trop tôt. Écrire `l’écran`, avec l’apostrophe typographique.
3. La source de `pile` n’a **pas d’accent grave de fermeture** à la fin : tout le reste du fichier serait pris pour du texte.

Dans les trois cas, l’atelier ne compile plus rien, et **F12** → **Console** affiche une erreur JavaScript qui donne le fichier et la ligne (section 0.10).

La recette : ajouter ta fonction à la console

●●● **avancé** on modifie le projet lui-même : prends ton temps

À lire avant : 0.10 Les touches utiles, et l'éditeur de texte · 0.11 · partie C · partie D

On va faire entrer `pile()` dans la console, exactement comme `bande()` y est entrée. La même recette marche pour **n'importe quelle fonction** écrite avec les fonctions qui existent déjà.

Ce qu'il te faut : un éditeur de texte (le Bloc-notes suffit ; Visual Studio Code est plus confortable) et le dossier du projet. On ne modifie **pas** le programme dans l'atelier, mais **trois fichiers du compilateur**, qui sont à côté d'`index.html`.

Étape 1 — L'écrire et l'essayer dans ton programme

Toujours commencer là : dans `principal.cpp`, ou dans un onglet voisin (`outils.cpp`). Tant qu'elle n'est pas parfaite, une fonction se corrige bien plus facilement dans l'atelier que dans le compilateur : l'écran se met à jour à chaque frappe.

Voici `pile()`, telle qu'elle est dans `outils.cpp` au chapitre du parcours :

`outils.cpp`

```
void pile(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t hauteur) {
    for (uint8_t i = 0; i < hauteur; i++) {    // hauteur fois :
        poser(colonne, ligne + i, tuile);      //   une case de plus vers le bas
    }
}
```

Étape 2 — Choisir son nom

Le nom du `#include` sera exactement celui de la fonction : `pile` → `#include <pile>`. Il doit être **libre** : aucune fonction de la console ne doit déjà le porter (la liste complète est en partie J).

Des lettres, des chiffres, le tiret bas `_` ; pas d'espace, pas d'accent, pas de chiffre au début. Les majuscules comptent : `Pile` et `pile` sont deux noms différents. L'habitude du projet : `motsCollesAvecMajuscules` (`cacherPanneau`) ou `mots_avec_tiret` (`deplace_x`).

Étape 3 — Le code : `compilateur/emetteur.js`

Ouvre `compilateur/emetteur.js` et cherche (Ctrl+F) `const SOURCE_BANDE` : c'est vers la ligne 5724. Voici, lue dans le fichier, la source de `bande()` :

`compilateur/emetteur.js`, ligne 5724

```
const SOURCE_BANDE = `
void bande(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i++) {    // longueur fois :
        poser(colonne + i, ligne, tuile);      //   une case de plus vers la droite
    }
}
```

```
}  
,
```

Lis-la morceau par morceau :

- `const SOURCE_BANDE =` : une **constante JavaScript** (le compilateur est écrit en JavaScript). Son nom est libre, mais l'habitude est `SOURCE_` + le nom en majuscules.
- L'**accent grave** (la touche AltGr + 7) ouvre un texte qui peut tenir sur plusieurs lignes. Un second accent grave le ferme, tout à la fin.
- Entre les deux : **ta fonction, en C, mot pour mot** comme dans `outils.cpp`.
- **Différence avec `outils.cpp` : pas de `#include <poser>`**. Ce que la console ajoute elle-même n'a rien à inclure.

Ajoute la tienne juste en dessous, avant le commentaire « Les fonctions de la console écrites en C » :

à ajouter dans `compilateur/emetteur.js`

```
const SOURCE_PILE = `  
void pile(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t hauteur) {  
    for (uint8_t i = 0; i < hauteur; i++) {    // hauteur fois :  
        poser(colonne, ligne + i, tuile);      //   une case de plus vers le bas  
    }  
}  
`  
,
```

Puis inscris-la dans le tableau `FONCTIONS_EN_C`, juste en dessous (vers la ligne 5766). Voici son début, lu dans le fichier :

`compilateur/emetteur.js`, ligne 5766

```
/** Les fonctions de la console écrites en C : leur nom, et leur texte. */  
const FONCTIONS_EN_C = {  
    bande: SOURCE_BANDE,
```

Ajoute une ligne, à côté de celle de `bande` :

```
    bande: SOURCE_BANDE,  
    pile: SOURCE_PILE,      ← NOUVEAU : le nom de la fonction, deux-points, la constante, virgule
```

Les petites erreurs qui cassent tout

Oublier la **virgule** au bout de la ligne du tableau. Oublier l'**accent grave** qui ferme la source. Écrire le nom du tableau (`pile`) autrement que le nom de la fonction dans la source (`void pile()`).

Dans ces cas, l'atelier ne peut plus rien compiler du tout : la page affiche une erreur JavaScript. Ouvre la console du navigateur (**F12**, onglet Console) : elle dit le fichier et la ligne.

Étape 4 — Le nom : `compilateur/inclusion.js`

Ouvre `compilateur/inclusion.js` et cherche `export const BIBLIOTHEQUES` (ligne 42). C'est la liste de tous les noms qu'on peut écrire entre chevrons, chacun avec une phrase qui dit ce qu'il fait. Voici la ligne de `bande`, lue dans le fichier (ligne 53) :

`compilateur/inclusion.js`, ligne 53

```
bande: 'pose la même tuile plusieurs fois, de gauche à droite',
```

Ajoute la tienne juste en dessous :

à ajouter dans compilateur/inclusion.js

```
pile: 'pose la même tuile plusieurs fois, vers le bas',
```

Cette ligne fait trois choses : elle rend `#include <pile>` valable ; elle oblige tout programme qui appelle `pile()` à écrire la ligne (sinon : refus) ; et sa phrase est recopiée en commentaire quand l'atelier écrit les `#include` à ta place (`#include <pile> // pose la même tuile...`).

La phrase est entre apostrophes droites `'...'`. Si ta phrase contient une apostrophe, prends l'apostrophe typographique `’` (comme dans « l'écran ») : une apostrophe droite fermerait le texte trop tôt.

Étape 5 — L'aide de l'éditeur : `aide-fonctions.js`

Ouvre `aide-fonctions.js` (à la racine du projet) et cherche `bande: {` (ligne 18) :

`aide-fonctions.js`, ligne 18

```
bande: { args: ['colonne', 'ligne', 'tuile', 'longueur'], dit: 'pose la même tuile « longueur » fois, de gauche à droite : bande(2, 5, ALPHABET[0], 10) écrit dix A sur la ligne 5.' },
```

à ajouter dans `aide-fonctions.js`

```
pile: { args: ['colonne', 'ligne', 'tuile', 'hauteur'], dit: 'pose la même tuile « hauteur » fois, vers le bas.' },
```

`args` : le nom de chaque argument, dans l'ordre. Pendant que tu tapes `pile(3,`, l'éditeur montre `pile(colonne, ligne, tuile, hauteur)` avec l'argument en cours en gras.

`dit` : la phrase affichée sous la proposition, quand tu tapes « pil... ».

Cette étape est facultative : sans elle, `pile()` marche, mais l'éditeur ne t'aide pas à l'écrire.

Étape 6 — Essayer

- **Recharge l'atelier avec Ctrl+F5** (pas F5 seul : Ctrl+F5 oblige le navigateur à relire les fichiers du compilateur au lieu de garder l'ancienne version en mémoire).
- Dans `outils.cpp`, **supprime** ta `pile()` (sinon, c'est la tienne qui compte, et tu n'essaies pas celle de la console).
- Dans `principal.cpp`, ajoute `#include <pile>` en haut, et lance.
- Pour vérifier le message d'oubli : enlève la ligne `#include <pile>` → le compilateur doit répondre « il faut `#include <pile>` ».
- **En ligne de commande** (une fenêtre ouverte dans le dossier du projet) : `node outils/gb3.mjs mon-essai.cpp` compile un fichier `.cpp` et dit s'il est accepté.
- **Puis** `npm run verifier` : les centaines de contrôles du projet. S'ils sont tous verts, tu n'as rien cassé.

Étape 7 — Sa leçon

Dans ce projet, chaque `#include` **a son tuto** : une leçon qui présente la fonction seule. Elle se met dans `tuto/lecons.js`. Le plus simple : cherche `La fonction bande()`, copie toute la leçon (de `{` à `}`), colle-la juste en dessous et adapte-la.

Le point important : la première ligne de son programme. C'est elle qui fait reconnaître la leçon comme le tuto de la fonction :

```
// ---- #include <pile> : la même tuile, vers le bas ----
```

Le parcours (`tuto/parcours.js`) place alors ce tuto **tout seul**, juste avant la première étape qui écrit `#include <pile>`. Le contrôle `npm run verifier` vérifie qu'aucune étape n'apporte deux `#include` nouveaux d'un coup.

Le **contrôle** de la leçon dit ce qu'on doit voir à l'écran, et le vérifie dans l'émulateur. Par exemple, pour six D en colonne 15, lignes 4 à 9 : `c.mot(15, 4, 1) === 'D'`.

Et pour aller plus loin : les fonctions en assembleur

Certaines fonctions de la console ne sont **pas** écrites en C, mais directement en instructions du processeur : celles qui parlent au matériel (l'écran, le son, la manette). Par exemple `cachePanneau()`, vers la ligne 4644 de `compilateur/emetteur.js` : trois instructions qui éteignent un bit du registre de l'écran.

Les ajouter demande de connaître le processeur de la Game Boy (un cousin du Z80) et ses registres. **Pour tout ce qu'on peut écrire avec les fonctions qui existent déjà, la façon en C de cette recette suffit**, et elle est bien plus sûre.

Les erreurs, et ce que le compilateur répond

●● **intermédiaire** il faut avoir lu ce qui est cité, et avoir essayé

À lire avant : 0.6 Une fonction : la définir, l'appeler · partie A

Chaque programme ci-dessous a été compilé pour écrire ce document, et le message est celui que l'atelier affiche, mot pour mot. Les reconnaître fait gagner beaucoup de temps.

1. On appelle bande() sans écrire #include <bande>

```
#include <ALPHABET>

int main() {
    bande(2, 5, ALPHABET[0], 10);
    while (true) { image(); }
}
```

LE COMPILATEUR RÉPOND

principal.cpp, ligne 4 : il faut « #include <bande> » pour employer bande(). Chaque fonction de la console prend de la place dans la cartouche : elle ne s'emploie qu'incluse. Écrire en haut du programme : #include <bande>

Pourquoi : C'est l'erreur la plus fréquente. Le compilateur connaît `bande()`, mais refuse de la graver sans qu'on la demande : il dit exactement quelle ligne écrire, et où.

Ce qu'il faut faire : Écrire `#include <bande>` en haut du programme.

2. Le nom est mal écrit : #include <Bande>

```
#include <ALPHABET>
#include <Bande>

int main() {
    bande(2, 5, ALPHABET[0], 10);
    while (true) { image(); }
}
```

LE COMPILATEUR RÉPOND

fichier introuvable : « Bande » : cette leçon n'a pas d'autre fichier.

Pourquoi : Le nom entre chevrons doit être écrit **exactement** comme la fonction, majuscules comprises. `Bande` n'est pas `bande` : n'étant pas une fonction de la console, le nom est cherché comme un FICHIER — et il n'y en a pas.

Ce qu'il faut faire : Écrire `#include <bande>`, en minuscules, comme dans `bande(...)`.

3. On demande une fonction que la console n'a pas : #include <pile>

```
#include <ALPHABET>
#include <pile>
```

```
int main() {
    pile(3, 4, ALPHABET[0], 8);
    while (true) { image(); }
}
```

LE COMPILATEUR RÉPOND

fichier introuvable : « pile » : cette leçon n'a pas d'autre fichier.

Pourquoi : `pile()` n'est pas (encore) dans `BIBLIOTHEQUES` : le compilateur ne la connaît pas comme fonction de la console, et cherche donc un fichier nommé `pile`. C'est le message qu'on obtient tant que l'étape 4 de la recette (le nom dans `compilateur/inclusion.js`) n'est pas faite.

Ce qu'il faut faire : Soit faire entrer `pile()` dans la console (partie E), soit l'écrire dans un fichier voisin et l'inclure avec des guillemets.

4. Le fichier voisin n'existe pas : `#include "outils.cpp"`

```
#include "outils.cpp"

int main() {
    while (true) { image(); }
}
```

LE COMPILATEUR RÉPOND

fichier introuvable : « outils.cpp » : cette leçon n'a pas d'autre fichier.

Pourquoi : Avec des guillemets, le compilateur cherche un ONGLET de ce nom à côté de `principal.cpp`. S'il n'y en a pas (ou si le nom diffère d'une lettre), il s'arrête.

Ce qu'il faut faire : Créer l'onglet avec le bouton « + fichier », et lui donner exactement le nom écrit entre les guillemets.

5. On appelle une fonction que personne n'a écrite

```
#include <ALPHABET>

int main() {
    pile(3, 4, ALPHABET[0], 8);
    while (true) { image(); }
}
```

LE COMPILATEUR RÉPOND

`principal.cpp`, ligne 4 : fonction inconnue « pile ». Celles de la console : `texte`, `poser`, `lire`, `image`, `bouton`, `hasard`, `semer`, `ecran`, `sprite`, `sprite16`, `sprite32`, `spriteTaille`, `cache`, `cache16`, `cache32`, `defiler`, `texteTitre`, `texteManga`, `panneau`, `cachePanneau`, `effacerPanneau`, `poserPanneau`, `lirePanneau`, `textePanneau`, `effacer`, `textS`, `poserS`, `attendre`, `couleurFond`, `couleurLutin`, `teindre`, `teindreLutin`, `teindrePanneau`, `paletteFond`, `paletteLutins`, `retard`, `images`, `note`, `bruit`, `silence`, `volumeSon`, `jouer`, `airFin`, `sauver`, `sauvegarde`, `nombre`, `nombrePanneau`, `changerDessin`. Les vôtres : `main`

Pourquoi : Ni le programme ni la console n'ont de `pile()`. Le compilateur donne la liste des fonctions de la console qu'il connaît, puis celles du programme (« les vôtres »).

Ce qu'il faut faire : Écrire la fonction (dans le programme ou un fichier voisin), ou l'ajouter à la console.

6. Il manque un argument

```
#include <ALPHABET>
#include <bande>

int main() {
    bande(2, 5, ALPHABET[0]);
    while (true) { image(); }
}
```

LE COMPILATEUR RÉPOND

principal.cpp, ligne 5 : « bande » prend 4 arguments, et non 3

Pourquoi : `bande()` en attend quatre : la colonne, la ligne, la tuile, la longueur. Le compilateur compte, et le dit.

Ce qu'il faut faire : Donner les quatre : `bande(2, 5, ALPHABET[0], 10);`

Ce que ça coûte dans la cartouche

●●● **avancé** on modifie le projet lui-même : prends ton temps
 À lire avant : 0.4 L'octet, et la place dans la cartouche · 0.5 · 0.6

Chaque programme a été compilé, et sa cartouche mesurée. Le nombre est la taille du programme compilé, en octets (une cartouche en tient 32 768).

Le programme	Octets	Par rapport au vide
Le programme vide : aucun <code>#include</code> , rien à l'écran.	71	—
<code>#include <bande></code> écrit, mais <code>bande()</code> jamais appelée.	71	—
Avec dix <code>poser()</code> écrits à la main (dix A).	229	+ 158
Un appel à <code>bande()</code> (les mêmes dix A).	911	+ 840
Trois appels à <code>bande()</code> .	949	+ 878

Ce que montre le tableau :

- Une ligne `#include` seule ne coûte rien : 71 octets, comme le programme vide. Le compilateur ne grave une fonction que si elle est **appelée**.
- Les appels suivants ne coûtent que quelques octets (38 octets pour deux appels de plus) : la fonction est gravée **une fois**, et chaque appel ne fait que lui donner ses quatre nombres.
- Mais un seul appel à `bande()` coûte bien plus que dix `poser()` (911 octets contre 229). Ce n'est pas la faute de `bande()` : le détail ci-dessous l'explique.

Le détail : où partent les octets

Le compilateur sait dire ce qu'il a gravé, morceau par morceau. Voici, pour les deux programmes qui posent dix A, la liste qu'il donne :

AVEC DIX `poser()` ÉCRITS À LA MAIN (DIX A). — 229 OCTETS

49	démarrage	la mise en route de la console
67	main()	ta fonction
8	VBlank	compte les images, 60 fois par seconde
28	AttendreImage	attend l'image suivante
16	AttendreVBlank	attend que l'écran soit libre
31	CopierTuiles	copie les lettres et les dessins en mémoire vidéo
14	EffacerCarte	vide les deux cartes de l'écran
16	Tuiles	1 lettre sur 43

UN APPEL À `bande()` (LES MÊMES DIX A). — 911 OCTETS

49	démarrage	la mise en route de la console
38	bande()	une fonction fournie par le langage
26	main()	ta fonction
8	VBlank	compte les images, 60 fois par seconde
28	AttendreImage	attend l'image suivante
16	AttendreVBlank	attend que l'écran soit libre
10	AdresseCase	trouve l'adresse d'une case de l'écran
18	CopierTuiles	copie les lettres et les dessins en mémoire vidéo
14	EffacerCarte	vide les deux cartes de l'écran
704	Tuiles	la police entière

Comment lire ces deux listes. Chaque ligne est un morceau gravé dans la cartouche : sa taille en octets, son nom, et son rôle. Les morceaux qu'on retrouve dans **tous** les programmes :

- `démarrage` : ce que la console fait à l'allumage, avant d'appeler `main()` (préparer l'écran, la mémoire).
- `main()` : **ta** fonction, traduite. Elle est plus courte avec `bande()` : un seul appel au lieu de dix.
- `VBlank`, `AttendreImage`, `AttendreVBlank` : la mesure du temps. La console dessine l'écran 60 fois par seconde ; `image()` attend le dessin suivant.
- `CopierTuiles`, `EffacerCarte` : copier les dessins des tuiles dans la mémoire de l'écran, et vider l'écran au départ.
- `AdresseCase` : la petite routine qui calcule où, dans la mémoire de l'écran, se trouve la case (colonne, ligne). `poser()` s'en sert.
- `Tuiles` : les **dessins** gravés, 16 octets par tuile (section 0.5).

`bande()` **elle-même ne pèse que 38 octets**. La différence est dans la ligne `Tuiles` : 16 octets d'un côté, 704 de l'autre.

Pourquoi. Avec `poser(2, 5, ALPHABET[0])`, la lettre est **écrite en clair** : le compilateur voit qu'on ne pose que des A, et ne grave **que le dessin du A** (16 octets : une tuile de 8 × 8 pixels). Avec `bande(2, 5, ALPHABET[0], 10)`, la lettre entre dans `bande()` par un **paramètre**, `tuile`, qui est une **variable** : à l'intérieur de la fonction, le compilateur ne peut plus savoir quelle lettre sera posée. Par prudence, il grave **toute la police**.

Ce qu'il faut en retenir. Ce coût est celui de la **police**, pas de la fonction : il est payé **une seule fois**, dès qu'une tuile passe par une variable quelque part dans le programme — les appels suivants ne le repaieront pas. C'est vrai aussi quand la tuile est un dessin à toi (une `Tuile`) : la police est gravée quand même, car le compilateur ne peut pas savoir ce que contiendra la variable.

Le bon réflexe : une fonction de la console qui reçoit une tuile en paramètre est pratique, mais pas gratuite. Si un programme doit tenir dans très peu de place, écrire les `poser()` en clair reste le plus économe.

Les questions qu'on se pose

●● **intermédiaire** il faut avoir lu ce qui est cité, et avoir essayé

À lire avant : partie A — Les deux sortes d'`#include` · partie E

Faut-il mettre `.cpp` ou `.h` à mon fichier voisin ?

Les deux marchent dans l'atelier : le nom n'est qu'un nom. L'habitude du C : un `.h` (« header ») pour des déclarations — des variables, des constantes, des dessins ; un `.cpp` pour des fonctions.

Où dois-je écrire la ligne `#include "outils.cpp"` ?

L'habitude : en haut, avec les autres `#include`. Pour des **fonctions**, l'ordre ne compte pas : le compilateur relève toutes les fonctions du programme avant de les traduire, donc `main()` peut appeler une fonction collée plus bas. Pour des **variables globales** et des **dessins**, l'ordre compte : ils doivent être versés **avant** la fonction qui s'en sert.

Un fichier voisin peut-il en inclure un autre ?

Oui : `outils.cpp` peut écrire `#include "dessins.h"`. Le collage se fait de proche en proche. Un fichier inclus deux fois n'est collé qu'une fois, et deux fichiers qui s'incluent l'un l'autre ne font pas tourner la console en rond.

Si j'écris `#include <bande>` deux fois, ou dans deux fichiers ?

Aucun problème : la fonction n'est gravée qu'une fois. Une ligne en double ne coûte rien.

J'ai écrit une fonction qui porte le nom d'une fonction de la console. Laquelle sert ?

La tienne. Le compilateur n'ajoute une fonction de la console que si le programme n'en a pas écrit une du même nom (leçon « Ta fonction passe avant celle de la console »). C'est pratique pour essayer une variante.

Ma fonction de console peut-elle se servir d'autres fonctions de la console ?

Oui, comme `bande()` se sert de `poser()`, et sans écrire leurs `#include` dans sa source. Le programme, lui, n'a à écrire que `#include <bande>`.

Ma fonction de console peut-elle rendre une valeur ?

Oui : au lieu de `void`, écris `uint8_t`, et termine par `return ...;`. `deplace_x()` en est un exemple : elle rend la colonne où la tuile s'est arrêtée.

Peut-elle avoir ses propres variables globales ?

Oui : la source peut déclarer des variables au-dessus de la fonction, dans le même texte. Choisis-leur un nom qui commence par celui de la fonction (`pile_compte`) pour ne pas gêner celles du programme.

J'ai modifié le compilateur et rien ne change dans l'atelier.

Le navigateur garde l'ancienne version en mémoire. **Ctrl+F5** l'oblige à tout relire. Si cela ne suffit pas, ferme l'onglet et rouvre l'atelier.

L'atelier ne compile plus rien du tout depuis ma modification.

Une faute de JavaScript dans un des trois fichiers (souvent une virgule ou un accent grave oublié). **F12**, onglet **Console** : le message donne le fichier et la ligne. Au pire, `git diff` montre ce que tu as changé, et `git checkout -- compilateur/emetteur.js` revient en arrière.

Mes changements seront-ils perdus si je mets le projet à jour ?

Ils sont dans les fichiers du projet, comme le reste. Fais un `git commit` après chaque fonction ajoutée : tu gardes une trace, et tu peux revenir en arrière.

L'aide-mémoire, à garder sous les yeux

●● **intermédiaire**

il faut avoir lu ce qui est cité, et avoir essayé

À lire avant : partie E — La recette : ajouter ta fonction à la console

Je veux...	J'écris...	Où
me servir d'une fonction de la console	<code>#include <nom></code>	en haut du programme
ranger mes fonctions à part	<code>#include "outils.cpp"</code>	en haut de <code>principal.cpp</code> ; l'onglet par « + fichier »
ajouter le code d'une fonction à la console	<code>const SOURCE_NOM = `...` + nom:</code> <code>SOURCE_NOM,</code>	<code>compilateur/emetteur.js</code>
rendre <code>#include <nom></code> valable	<code>nom: 'ce qu'elle fait',</code>	<code>compilateur/inclusion.js</code>
que l'éditeur la propose	<code>nom: { args: [...], dit: '...' },</code>	<code>aide-fonctions.js</code>
lui donner sa leçon	<code>// ---- #include <nom> : ... ----</code>	<code>tuto/lecons.js</code>
tout vérifier	<code>npm run verifier</code>	une fenêtre dans le dossier du projet

À cocher, pour chaque fonction ajoutée

- ☐ La fonction marche dans mon programme (ou dans `outils.cpp`).
- ☐ Son nom est libre (absent de la partie J) et écrit pareil partout.
- ☐ `compilateur/emetteur.js` : `const SOURCE_...` ajoutée, sans `#include` dedans, l'accent grave fermé.
- ☐ `compilateur/emetteur.js` : une ligne `nom: SOURCE_...`, dans `FONCTIONS_EN_C`, avec sa virgule.
- ☐ `compilateur/inclusion.js` : une ligne `nom: 'ce qu'elle fait',` dans `BIBLIOTHEQUES`.
- ☐ `aide-fonctions.js` : une ligne `nom: { args: [...], dit: '...' },` dans `FONCTIONS` (facultatif).
- ☐ Ctrl+F5 dans l'atelier ; ma version retirée d' `outils.cpp` ; `#include <nom>` écrit ; ça marche.
- ☐ Sans la ligne `#include <nom>`, le compilateur refuse : la règle est en place.
- ☐ `npm run verifier` : tout est vert.
- ☐ Sa leçon dans `tuto/lecons.js`, programme commençant par `// ---- #include <nom> : ... ----`.
- ☐ `git commit`.

Tous les #include de la console

• **débutant** aucune connaissance demandée en dehors de ce qui est cité

Les 81 noms qu'on peut écrire entre chevrons aujourd'hui, lus dans `compilateur/inclusion.js`. Un nouveau nom doit être **absent** de cette liste.

#include	Ce qu'il apporte
<code>#include <texte></code>	écrit un texte à l'écran
<code>#include <textS></code>	écrit un texte qui passe à la ligne tout seul
<code>#include <texteGrand></code>	écrit un texte agrandi, taille de 0 à 10
<code>#include <texteGrandS></code>	écrit un texte agrandi qui passe à la ligne
<code>#include <texteCouleur></code>	écrit un mot dans une palette de couleur
<code>#include <nombre></code>	écrit un nombre en chiffres
<code>#include <effacer></code>	efface des cases, ou tout le fond
<code>#include <poser></code>	pose une tuile sur une case du fond
<code>#include <poserS></code>	pose une tuile, et passe à la ligne au bord
<code>#include <bande></code>	pose la même tuile plusieurs fois, de gauche à droite
<code>#include <poserDevant></code>	pose une tuile qui passe devant les lutins (Game Boy Color)
<code>#include <lire></code>	lit la tuile posée sur une case
<code>#include <changerDessin></code>	change le dessin d'une tuile partout à la fois
<code>#include <defiler></code>	fait glisser tout le fond
<code>#include <ecran></code>	éteint ou rallume l'écran
<code>#include <panneau></code>	montre le panneau, à une place choisie
<code>#include <cacherPanneau></code>	cache le panneau
<code>#include <effacerPanneau></code>	efface des cases du panneau, ou tout le panneau
<code>#include <textePanneau></code>	écrit un texte sur le panneau
<code>#include <nombrePanneau></code>	écrit un nombre sur le panneau
<code>#include <poserPanneau></code>	pose une tuile sur le panneau
<code>#include <lirePanneau></code>	lit une tuile du panneau
<code>#include <ALPHABET></code>	les lettres de la police : ALPHABET[0] est le A
<code>#include <ALPHABET_GRAS></code>	l'alphabet en gras : ALPHABET_GRAS[0] est le A gras
<code>#include <ALPHABET_TITRE></code>	l'alphabet des titres : lettres épaisses, avec une ombre ; ALPHABET_TITRE[0] est le A
<code>#include <texteTitre></code>	écrit un mot en GROSSES lettres de titre (taille 2, 3 ou 4 cases de côté)

#include	Ce qu'il apporte
#include <texteManga>	écrit un mot en lettres de titre MANGA : penchées, coins coupés, une trame (taille 2, 3 ou 4)
#include <Tuile>	un dessin de 8 × 8 pixels
#include <Perso>	un personnage : un dessin de 16 × 16 ou de 32 × 32 pixels
#include <Grand>	un autre nom pour un Perso de 32 × 32 pixels
#include <Mot>	un texte et sa place, sous un seul nom
#include <sprite>	place un lutin de 8 × 8 au pixel près
#include <spriteDerriere>	place un lutin de 8 × 8 derrière le décor
#include <sprite16>	place un lutin de 16 × 16 au pixel près
#include <sprite32>	place un grand personnage de 32 × 32 au pixel près
#include <spriteTaille>	place un dessin à la taille qu'on veut, sans limite : dessiné une fois, agrandi par le compilateur
#include <cache>	cache un lutin
#include <cache16>	cache un lutin de 16 × 16
#include <cache32>	cache un grand personnage de 32 × 32
#include <couleurFond>	choisit une couleur d'une palette du fond
#include <couleurTexte>	choisit la couleur des lettres
#include <couleurLutin>	choisit une couleur d'une palette des lutins
#include <teindre>	met une case du fond dans une palette
#include <teindrePanneau>	met une case du panneau dans une palette
#include <teindreLutin>	met un lutin dans une palette
#include <paletteFond>	choisit les quatre nuances du fond
#include <paletteLutins>	choisit les quatre nuances des lutins
#include <deplace_x>	fait avancer une tuile sur sa ligne
#include <deplace_y>	fait avancer une tuile sur sa colonne
#include <deplace>	fait avancer une tuile sur les deux axes
#include <va_a>	mène une tuile jusqu'à une case
#include <un_pas>	fait faire un seul pas à une tuile
#include <vitesse>	règle la vitesse des déplacements
#include <deplace_croix>	une tuile qui suit la croix, case par case
#include <glisse_croix>	un lutin qui suit la croix, au pixel près
#include <carre>	une tuile qui tourne en carré
#include <Carre>	les sept réglages d'un carré sous un seul nom

#include	Ce qu'il apporte
#include <losange>	une tuile qui tourne en losange
#include <rectangle>	une tuile qui tourne en rectangle
#include <spirale>	une tuile qui tourne en spirale
#include <aller_retour>	une tuile qui va et revient
#include <tourne_carre>	une tuile qui tourne en carré sans arrêter le jeu
#include <defile>	une tuile qui file sur sa ligne sans arrêter le jeu
#include <chaque>	répond 1 toutes les n millisecondes
#include <attendre>	attend des secondes entières
#include <bouton>	lit un bouton de la manette
#include <hasard>	tire un nombre au hasard
#include <semer>	choisit le départ du hasard
#include <note>	joue une note
#include <bruit>	joue un bruit
#include <silence>	fait taire une voix
#include <volumeSon>	règle le volume général
#include <Air>	un air de musique, note par note
#include <jouer>	joue un air tout seul
#include <airFini>	dit si un air est fini
#include <sauver>	garde un nombre dans la cartouche, même éteinte
#include <sauvegarde>	relit un nombre gardé dans la cartouche
#include <multiplier>	$a * b$, quand les deux se calculent
#include <diviser>	a / b , sauf par 1, 2, 4, 8, 16... écrits en clair
#include <reste>	$a \% b$, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <decaler>	$a << b$ et $a >> b$, quand b se calcule

Le lexique : chaque mot technique

• **débutant** aucune connaissance demandée en dehors de ce qui est cité

Les 50 mots techniques du document, par ordre alphabétique. La dernière colonne renvoie là où le mot est expliqué en détail (cliquable).

accent grave	Le signe ` (AltGr+7). En JavaScript, il entoure un texte qui peut tenir sur plusieurs lignes.	partie D
accolades	Les signes { et }. Elles regroupent des ordres (C++) ou les éléments d'un objet (JavaScript).	0.8
ALPHABET	Le tableau des tuiles de la police : ALPHABET[0] est le A, ALPHABET[1] le B...	0.5
appel	Le moment où l'on fait le geste d'une fonction : bande(2, 5, ALPHABET[0], 10);	0.6
argument	La valeur donnée à un paramètre au moment de l'appel : dans bande(2, ...), 2 est un argument.	0.6
BIBLIOTHEQUES	La liste, dans compilateur/inclusion.js, de tous les noms qu'on peut écrire entre chevrons.	partie E
bit	Un interrupteur qui vaut 0 ou 1. Huit bits font un octet.	0.4
boucle	Des ordres répétés. La boucle for les répète en comptant.	0.7
cartouche	Le fichier .gb fabriqué par le compilateur : tout le jeu, en nombres. 32 768 octets au plus.	0.3
case	Un carré de 8 × 8 pixels de l'écran, désigné par sa colonne (0 à 19) et sa ligne (0 à 17).	0.5
chemin	Où trouver un fichier, dossier par dossier : compilateur/inclusion.js.	0.2
chevrons	Les signes < et >. Autour d'un nom dans #include, ils désignent une fonction de la console.	partie A
commentaire	Une explication pour l'humain, ignorée par la machine : après // ou entre /* et */.	0.8
compilateur	Le traducteur qui transforme ton C++ en nombres pour le processeur.	0.3
console (fonction de la)	Une fonction écrite par le projet (poser, texte, bande...), qu'on demande par #include <nom>.	partie A
const	En JavaScript : donner un nom fixe à une valeur.	partie D
corps	Les ordres d'une fonction ou d'une boucle, entre accolades.	0.6
deLaConsole	La marque que le compilateur pose sur les fonctions qu'il ajoute lui-même : elles n'ont rien à inclure.	partie B
dossier	Une boîte qui contient des fichiers et d'autres dossiers.	0.2
éditeur de texte	Un programme qui ouvre et enregistre un fichier tel quel (Bloc-notes, Visual Studio Code). Pas Word.	0.10
émulateur	Une Game Boy imitée par l'ordinateur : il lance la cartouche.	0.3

expression régulière	Un motif qui reconnaît la forme d'un texte, comme un pochoir.	partie B
extension	La fin du nom d'un fichier, après le point : .cpp, .js, .pdf.	0.2
fichier	Un document rangé sur l'ordinateur. Dans l'atelier, un onglet de l'éditeur.	0.2
fichier voisin	Un fichier à toi, à côté de principal.cpp, qu'on colle par #include "nom".	partie A
fonction	Un geste auquel on a donné un nom, pour le refaire sans le réécrire.	0.6
 Fonctions_en_C	L'objet, dans compilateur/emetteur.js, qui relie le nom de chaque fonction de la console écrite en C++ à sa source.	partie E
for	La boucle qui compte : for (départ ; tant que ; après chaque tour).	0.7
git	Le programme qui photographie le projet, pour pouvoir revenir en arrière.	0.12
guillemets	Le signe ". Autour d'un nom dans #include, ils désignent un fichier à toi.	partie A
#include	La ligne qui dit « ce programme a besoin de quelque chose qui n'est pas écrit ici ».	partie A
JavaScript	Le langage de l'atelier et du compilateur, que les navigateurs savent faire tourner.	partie D
main()	La fonction par laquelle tout programme commence.	0.6
node	Le programme qui fait tourner du JavaScript dans une fenêtre de commande.	0.11
objet	En JavaScript : une liste de « nom: valeur, » entre accolades.	partie D
octet	Une case de mémoire qui retient un nombre de 0 à 255.	0.4
paramètre	La case nommée, dans la définition d'une fonction, que l'appel remplit.	0.6
pixel	Un point de l'écran. L'écran en a 160 × 144.	0.5
point-virgule	Le signe ; qui termine un ordre en C++.	0.8
police	L'ensemble des tuiles des lettres et des chiffres.	0.5
principal.cpp	Le fichier principal du programme, celui par lequel la compilation commence.	0.9
processeur	La puce de la Game Boy qui exécute les ordres, un par un.	0.3
return	L'ordre qui rend une valeur et termine la fonction.	0.6
source	Le texte d'un programme, tel qu'on l'écrit. SOURCE_BANDE est la source de bande().	partie E
tableau	Une liste de valeurs rangées à la suite ; on désigne chacune par sa place entre crochets.	0.5
terminal	La fenêtre de commande, où l'on tape des ordres au clavier.	0.11
tuile	Le dessin d'une case : 8 × 8 pixels, 16 octets.	0.5
type	Le genre de valeur qu'une case retient. Ici presque toujours uint8_t.	0.6
uint8_t	Le type « octet » : un nombre entier de 0 à 255.	0.4
void	« Vide » : la fonction ne rend rien.	0.6